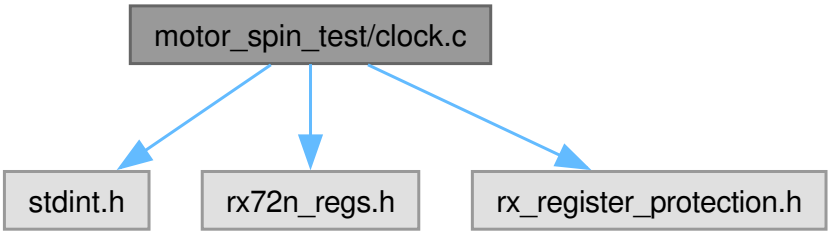
>>

>>

>>

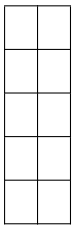
```
#include <stdint.h>
#include "rx72n_regs.h"
#include "rx_register_protection.h"
```



>>

>>>

enum clock_byte_constants_t : uint8_t



```
00060             : uint8_t {
00061   k_sub_clock_stopped = 0x01U, /**< SOSCCR: stop sub-clock oscillator */
00062   k_main_osc_enabled  = 0x00U, /**< MOSCCR: 0 = MOSC running */
00063   k_pll_enabled       = 0x00U, /**< PLLCR2: 0 = PLL running */
00064   k_oscovfsr_moovf    = 0x01U, /**< OSCOVFSR bit 0: MOSC stable */
00065   k_oscovfsr_plovf    = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00066 } clock_byte_constants_t;
```

enum clock_extra_addrs_t : uintptr_t

```
00029             : uintptr_t {
00030   k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL */
00031 } clock_extra_addrs_t;
```

enum clock_sckcr_t : uint32_t

```
00051             : uint32_t {
00052   /*
00053    * SCKCR dividers (bench-test config, ICLK half-speed vs production):
00054    *   FCK=/4 (60), ICK=/2 (120), PSTOP0=1, PSTOP1=1, BCK=/4 (60),
00055    *   PCKA=/2 (120), PCKB=/4 (60), PCKC=/2 (120), PCKD=/2 (120).
00056    */
00057   k_sckcr_value = 0x21C21211U,
00058 } clock_sckcr_t;
```

enum clock_timeout_t : uint32_t

```
00068             : uint32_t {
00069   /* Bounded poll counts (NASA Rule 2). We poll the hardware ready flag
00070    * (OSCOVFSR.MOOVF / .PLOVF) so the actual wait is hardware-determined --
00071    * roughly 10 ms physical time regardless of CPU clock. The upper bound
00072    * just prevents an infinite spin if the crystal is missing. ~500k iters
00073    * at LOCO 240 kHz is ~3 s, which is still much shorter than the prior
00074    * fixed 10-second busy-wait. */
00075   k_main_osc_poll_max  = 500000U,
00076   k_pll_lock_poll_max  = 500000U,
00077 } clock_timeout_t;
```

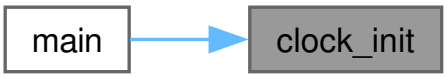
enum clock_word_constants_t : uint16_t

```
00036             : uint16_t {
00037   /*
00038    * PLLCR layout:
00039    *   bits[1:0]  PLIDIV    : 00 = /1
00040    *   bit [4]    PLLSRCSEL : 0 = MOSC, 1 = HOCO
00041    *   bits[13:8] STC       : multiplier = (STC + 1) / 2
00042    *
00043    * MOSC 24 MHz / 1 * 10 = 240 MHz. STC = (10 * 2) - 1 = 19 = 0x13.
00044    * PLLSRCSEL = 0 (MOSC) -> bit 4 clear.
00045    *
00046    * Production rx_clock_power_init.c uses k_pll_multiplier _div  = 0x1300.
00047    */
00048   k_pllcr_mosc_x10 = 0x1300U,
00049 } clock_word_constants_t;
```

void clock_init (
                  void)

```
00083 {
00084   volatile rx_system_regs_t *sys = system_regs();
00085
00086   /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
00087   *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00088
00089   /* Stop sub-clock oscillator -- not used on STAR. */
00090   sys->sosccr = k_sub_clock_stopped;
00091
00092   /* Start MOSC (24 MHz external crystal). */
00093   sys->mosccr = k_main_osc_enabled;
00094
00095   /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
00096    * settled (typically ~10 ms physical time) instead of waiting a fixed
00097    * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
00098   for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
00099     if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
00100       break;
00101     }
00102   }
00103
00104   /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
00105   sys->pllcr  = k_pllcr_mosc_x10;
00106   sys->pllcr2 = k_pll_enabled;
00107
00108   /* Wait for PLL lock (bounded). If we time out, the crystal is missing
00109    * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
00110    * on a dead clock. Stay on LOCO so the LEDs still respond and the user
00111    * can see we got past clock_init. */
00112   bool pll_locked = false;
00113   for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
00114     if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
00115       pll_locked = true;
00116       break;
00117     }
00118   }
00119
00120   if (pll_locked) {
00121     /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
00122     *memwait_reg() = k_memwait_one_wait;
00123
00124     /* Apply system clock dividers. */
00125     sys->sckcr = k_sckcr_value;
00126
00127     /* Route UCLK from main PLL (UPLLSEL=0). */
00128     *((volatile uint16_t *)k_packcr_addr) = 0x0000U;
00129
00130     /* Switch system clock source to PLL. */
00131     sys->sckcr3 = k_sckcr3_cksel_pll;
00132   }
00133
00134   /* Re-lock PRCR. */
```

```
00135   *prcr_reg() = k_rx_prcr_lock;
00136 }
```




```
00001 /**
00002  * @file clock.c
00003  * @brief RX72N clock init for the STAR PCB bench test: MOSC 24 MHz crystal
00004  *        -> PLL 240 MHz -> ICLK=120 / PCKA=120 / PCKB=60 / FCLK=60 MHz.
00005  *
00006  * @details
00007  * Self-contained reduced version of src/rx_clock_power_init.c, stripped of
00008  * the logging, asserts, simulator branches, and PPLL/USB clock paths the
00009  * motor spin test does not need. Configures only the main PLL.
00010  *
00011  * Path: EXTAL 24 MHz -> MOSC -> PLL (x10 / 1) = 240 MHz -> SCKCR=0x21C21211.
00012  * After this routine returns, ICLK is 120 MHz (half of production's 240 MHz
00013  * -- the bench test runs the CPU at reduced speed to cut power dissipation)
00014  * and PCLKA (the GPTW source) is 120 MHz, matching k_pclka_hz in
00015  * libs/rx_hal/inc/rx72n_clock.h.
00016  *
00017  * SPDX-License-Identifier: MIT
00018  * @copyright Copyright (c) 2026 Locked Inc.
00019  */
00020
00021 #include <stdint.h>
00022
00023 #include "rx72n_regs.h"
00024 #include "rx_register_protection.h"
00025
00026 /*
      ============================================================================
00027  * Register addresses not exposed by rx_system_regs_t
00028  *
      ============================================================================
      */
00029 typedef enum : uintptr_t {
00030   k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL */
00031 } clock_extra_addrs_t;
00032
00033 /*
      ============================================================================
00034  * Bit / value constants (mirrors src/rx_clock_power_init.c)
00035  *
      ============================================================================
      */
00036 typedef enum : uint16_t {
00037   /*
00038    * PLLCR layout:
00039    *   bits[1:0]  PLIDIV    : 00 = /1
00040    *   bit [4]    PLLSRCSEL : 0 = MOSC, 1 = HOCO
00041    *   bits[13:8] STC       : multiplier = (STC + 1) / 2
00042    *
00043    * MOSC 24 MHz / 1 * 10 = 240 MHz. STC = (10 * 2) - 1 = 19 = 0x13.
00044    * PLLSRCSEL = 0 (MOSC) -> bit 4 clear.
00045    *
00046    * Production rx_clock_power_init.c uses k_pll_multiplier_div  = 0x1300.
00047    */
00048   k_pllcr_mosc_x10 = 0x1300U,
00049 } clock_word_constants_t;
00050
00051 typedef enum : uint32_t {
00052   /*
00053    * SCKCR dividers (bench-test config, ICLK half-speed vs production):
00054    *   FCK=/4 (60), ICK=/2 (120), PSTOP0=1, PSTOP1=1, BCK=/4 (60),
00055    *   PCKA=/2 (120), PCKB=/4 (60), PCKC=/2 (120), PCKD=/2 (120).
00056    */
00057   k_sckcr_value = 0x21C21211U,
00058 } clock_sckcr_t;
00059
```

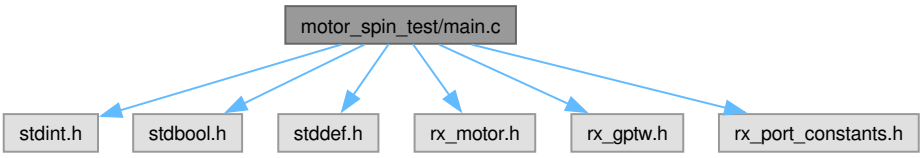
```
00060 typedef enum : uint8_t {
00061   k_sub_clock_stopped = 0x01U, /**< SOSCCR: stop sub-clock oscillator */
00062   k_main_osc_enabled  = 0x00U, /**< MOSCCR: 0 = MOSC running */
00063   k_pll_enabled       = 0x00U, /**< PLLCR2: 0 = PLL running */
00064   k_oscovfsr_moovf    = 0x01U, /**< OSCOVFSR bit 0: MOSC stable */
00065   k_oscovfsr_plovf    = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00066 } clock_byte_constants_t;
00067
00068 typedef enum : uint32_t {
00069   /* Bounded poll counts (NASA Rule 2). We poll the hardware ready flag
00070    * (OSCOVFSR.MOOVF / .PLOVF) so the actual wait is hardware-determined --
00071    * roughly 10 ms physical time regardless of CPU clock. The upper bound
00072    * just prevents an infinite spin if the crystal is missing. ~500k iters
00073    * at LOCO 240 kHz is ~3 s, which is still much shorter than the prior
00074    * fixed 10-second busy-wait. */
00075   k_main_osc_poll_max  = 500000U,
00076   k_pll_lock_poll_max  = 500000U,
00077 } clock_timeout_t;
00078
00079 /*
      ============================================================================
00080  * clock_init -- bring the chip from POR defaults to PLL 240 / PCKA 120 MHz.
00081  *
      ============================================================================
      */
00082 void clock_init(void)
00083 {
00084   volatile rx_system_regs_t *sys = system_regs();
00085
00086   /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
00087   *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00088
00089   /* Stop sub-clock oscillator -- not used on STAR. */
00090   sys->sosccr = k_sub_clock_stopped;
00091
00092   /* Start MOSC (24 MHz external crystal). */
00093   sys->mosccr = k_main_osc_enabled;
00094
00095   /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
00096    * settled (typically ~10 ms physical time) instead of waiting a fixed
00097    * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
00098   for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
00099     if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
00100       break;
00101     }
00102   }
00103
00104   /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
00105   sys->pllcr  = k_pllcr_mosc_x10;
00106   sys->pllcr2 = k_pll_enabled;
00107
00108   /* Wait for PLL lock (bounded). If we time out, the crystal is missing
00109    * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
00110    * on a dead clock. Stay on LOCO so the LEDs still respond and the user
00111    * can see we got past clock_init. */
00112   bool pll_locked = false;
00113   for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
00114     if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
00115       pll_locked = true;
00116       break;
00117     }
00118   }
00119
00120   if (pll_locked) {
00121     /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
00122     *memwait_reg() = k_memwait_one_wait;
00123
00124     /* Apply system clock dividers. */
00125     sys->sckcr = k_sckcr_value;
00126
00127     /* Route UCLK from main PLL (UPLLSEL=0). */
00128     *((volatile uint16_t *)k_packcr_addr) = 0x0000U;
00129
00130     /* Switch system clock source to PLL. */
00131     sys->sckcr3 = k_sckcr3_cksel_pll;
00132   }
00133
00134   /* Re-lock PRCR. */
00135   *prcr_reg() = k_rx_prcr_lock;
00136 }
```

```
00001 /*
```

```
00002 * gpio_test/linker.ld -- Linker script for GPIO breakout board test.
00003 *
00004 * Same memory map as usb_test: 512 KB internal SRAM, 2 MB internal ROM.
00005 * Includes Renesas-syntax sections (P, B, D, C) needed by ThreadX RXv3 port.
00006 */
00007
00008 MEMORY
00009 {
00010     RAM : ORIGIN = 0x00000004, LENGTH = 0x7FFFC   /* 512 KB - 4 bytes */
00011     ROM : ORIGIN = 0xFFE00000, LENGTH = 0x200000  /* 2 MB             */
00012 }
00013
00014 SECTIONS
00015 {
00016     /* Code -- explicit ROM base so section VMA is anchored correctly. */
00017     .text 0xFFE00000 : AT(0xFFE00000)
00018     {
00019         *(.text*)
00020         *(P)
00021         *(P )
00022         *(.rodata*)
00023         *(C)
00024         *(C )
00025         *(C )
00026         etext = .;
00027     } > ROM
00028
00029     /*
00030      * Relocatable vector table -- startup.S loads INTB with this address.
00031      * Must be 4-byte aligned; only vectors 27 (SWINT) and 28 (CMT0) are
00032      * used, but allocate 256 entries (1024 bytes) for safety.
00033      */
00034     .rvectors ALIGN(4) :
00035     {
00036         __rvectors_start = .;
00037         KEEP(*(.rvectors))
00038         __rvectors_end = .;
00039     } > ROM
00040
00041     /*
00042      * Initialised data -- LMA stays in ROM, VMA in RAM.
00043      * startup.S copies _data..edata from _mdata on boot.
00044      */
00045     _mdata = .;
00046     .data : AT(_mdata)
00047     {
00048         _data = .;
00049         *(.data*)
00050         *(D)
00051         *(D )
00052         *(D )
00053         _edata = .;
00054     } > RAM
00055
00056     /* BSS -- zeroed by startup.S */
00057     .bss :
00058     {
00059         _bss = .;
00060         *(.bss*)
00061         *(COMMON)
00062         *(B)
00063         *(B )
00064         *(B )
00065         _ebss = .;
00066         . = ALIGN(4);
00067         _end = .;
00068     } > RAM AT>RAM
00069
00070     /*
00071      * Stacks -- USP grows down from top of RAM, ISP 4 KB below that.
00072      */
00073     _ustack = ORIGIN(RAM) + LENGTH(RAM);
00074     _istack = _ustack - 0x1000;
00075
00076     /*
00077      * Exception vector table at fixed hardware address 0xFFFFFF80.
00078      * 32 entries -- all unused.
00079      */
00080     .exvectors 0xFFFFFF80 : AT(0xFFFFFF80)
00081     {
00082         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00083         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00084         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00085         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00086         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00087         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00088         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
```

```
00089         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00090     } > ROM
00091
00092     /* Fixed reset vector at 0xFFFFFFFC */
00093     .fvectors 0xFFFFFFFC : AT(0xFFFFFFFC)
00094     {
00095         KEEP(*(.fvectors))
00096     } > ROM
00097 }
```

```
#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>
#include "rx_motor.h"
#include "rx_gptw.h"
#include "rx_port_constants.h"
```



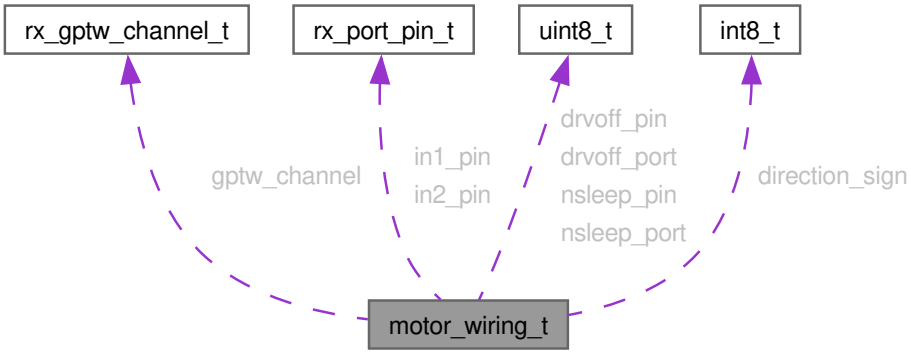
<<

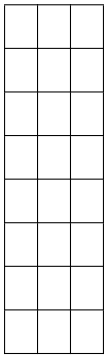
<<

*

*
*
*

****





#define MOTOR_DUTY_MAX (100)

#define MOTOR_DUTY_MIN (-100)

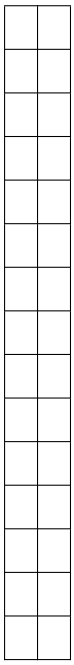
#define MOTOR_ENABLE_MASK 0xFU

#define REG16(
a)

(*(volatile uint16_t *)(uintptr_t)(a))

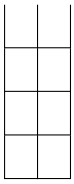
```
#define REG8(
                        a)


 (*(volatile uint8_t *)(uintptr_t)(a))
```

```
enum adc_addrs_t : uintptr_t
```



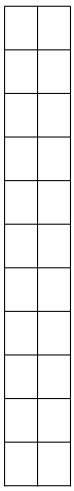
```
00310             : uintptr_t {
00311   k_s12ad0_adcsr_addr   = 0x00089000U,
00312   k_s12ad0_adansa0_addr = 0x00089004U,
00313   k_s12ad0_adcer_addr   = 0x0008900EU,
00314   k_s12ad0_adsstr4_addr = 0x000890E4U,
00315   k_s12ad0_addr4_addr   = 0x00089028U, /* AN004 result */
00316   k_s12ad0_addr5_addr   = 0x0008902AU,
00317   k_s12ad0_addr6_addr   = 0x0008902CU,
00318   k_s12ad0_addr7_addr   = 0x0008902EU,
00319   k_mstpcra_addr        = 0x00080010U,
00320   k_prcr_addr           = 0x000803FEU,
00321   k_mpc_pwpr_addr       = 0x0008C11FU,
00322   k_mpc_p44pfs_addr     = 0x0008C124U,
00323   k_mpc_p45pfs_addr     = 0x0008C125U,
00324   k_mpc_p46pfs_addr     = 0x0008C126U,
00325   k_mpc_p47pfs_addr     = 0x0008C127U,
00326 } adc_addrs_t;
```

```
enum adc_cfg16_t : uint16_t
```



```
00328             : uint16_t {
00329   k_adcsr_adst        = (uint16_t)(1U « 15),
00330   k_adansa_ch4567     = (uint16_t)0x00F0U, /* enable AN004..AN007 */
00331   k_adcer bit_right = (uint16_t)0x0000U, /* ADPRC=00, ADRFMT=0 */
00332   k_prcr_unlock       = (uint16_t)0xA50BU, /* unlock PRC0+PRC1+PRC3 */
00333   k_prcr_lock         = (uint16_t)0xA500U,
00334 } adc_cfg16_t;
```

enum adc_cfg8_t : uint8_t



```
00340             : uint8_t {
00341   k_pfs_asel     = 0x80U,
00342   k_pwpr_unlock  = 0x00U,
00343   k_pwpr_pfswe   = 0x40U,
00344   k_pwpr_b0wi    = 0x80U,
00345   k_adsstr_def   = 0x14U, /* 20 states sampling, conservative */
00346   /* ADSSTR channel offsets in bytes from ADSSTR4 base (one per channel). */
00347   k_adsstr_off_ch4 = 0U,
00348   k_adsstr_off_ch5 = 1U,
00349   k_adsstr_off_ch6 = 2U,
00350   k_adsstr_off_ch7 = 3U,
00351   /* Decimal print helpers. */
00352   k_u16_dec_max_digits = 5U,  /* 65535 is 5 digits */
00353   k_dec_radix          = 10U,
00354 } adc_cfg8_t;
```

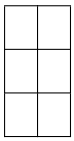
enum adc_mstp_t : uint32_t

```
00336             : uint32_t {
00337   k_mstpa17 = (uint32_t)(1UL « 17), /* S12AD0 stop bit */
00338 } adc_mstp_t;
```

enum adc_poll_t : uint32_t

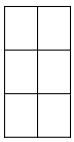
```
00356             : uint32_t {
00357   /* Bound on polling loop waiting for ADCSR.ADST to self-clear; the actual
00358    * conversion completes in <5 us even at 60 MHz PCLKB, so 100k iters is
00359    * pure safety against a hung peripheral (~400 us wall-clock @ 240 MHz). */
00360   k_adc_adst_poll_max = 100000U,
00361 } adc_poll_t;
```

enum delay_iters_t : uint32_t



```
00176             : uint32_t {
00177   /* ~40 ms per duty step at ICLK=240 MHz, -O1; tuned coarsely. */
00178   k_step_delay_iters = 1000000U,
00179   /* ~10 ms tWAKE margin after nSLEEP rises (DRV8263H spec: 1.2 ms min). */
00180   k_twake_iters = 250000U,
00181   /* Split each duty-step dwell around the ADC scan (half before, half after). */
00182   k_step_delay_halves = 2U,
00183 } delay_iters_t;
```

enum duty_sweep_t : int8_t

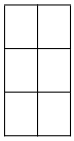


```
00199             : int8_t {
00200   k_duty_min     = MOTOR_DUTY_MIN,
00201   k_duty_max     = MOTOR_DUTY_MAX,
00202   k_duty_step_pc = 5,
00203 } duty_sweep_t;
```

enum motor_count_t : uint8_t

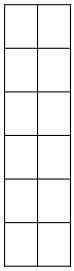
```
00120             : uint8_t {
00121   k_motor_count = 4,
00122 } motor_count_t;
```

enum port__reg__base__t : uintptr_t



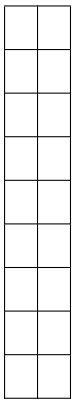
```
00059            : uintptr_t {
00060   k_port__pdr__base  = 0x0008C000U,
00061   k_port__podr__base = 0x0008C020U,
00062   k_port__pmr__base  = 0x0008C060U,
00063 } port__reg__base__t;
```

enum rx__port__index__t : uint8_t



```
00095            : uint8_t {
00096   k_port  = 1,
00097   k_port  = 2,
00098   k_port  = 6,
00099   k_port  = 8,
00100   k_port_c = 12,
00101   k_port_e = 14,
00102 } rx__port__index__t;
```

enum status__led__pins__t : uint8_t

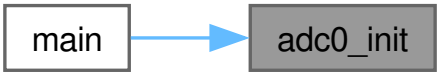


```
00081            : uint8_t {
00082   k_port_a   = 10, /* PORTA */
00083   k_port_b   = 11, /* PORTB */
00084   k_port    = 7,  /* PORT7 */
00085   k_bit_led0 = 7,  /* PA7 */
```

```
00086   k_bit_led1 = 0,  /* PB0 */
00087   k_bit_led2 = 1,  /* P71 */
00088   k_bit_led3 = 2,  /* P72 */
00089   k_bit_led4 = 1,  /* PB1 */
00090   k_bit_led5 = 2,  /* PB2 */
00091 } status_led_pins_t;
```

```
void adc0_init (
                void )    [static]
```

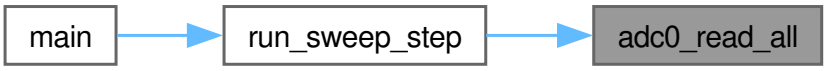
```
00364 {
00365   /* 1. Release S12AD0 from module-stop (MSTPA17 in MSTPCRA). */
00366   REG16(k_prcr_addr) = k_prcr_unlock;
00367   *(volatile uint32_t *)k_mstpcra_addr &= ~(uint32_t)k_mstpa17;
00368   REG16(k_prcr_addr) = k_prcr_lock;
00369
00370   /* 2. Set PFS.ASEL=1 for P44-P47 (analog input, digital buffer off). */
00371   *(volatile uint8_t *)k_mpc_pwpr_addr    = k_pwpr_unlock;
00372   *(volatile uint8_t *)k_mpc_pwpr_addr    = k_pwpr_pfswe;
00373   *(volatile uint8_t *)k_mpc_p44pfs_addr  = k_pfs_asel;
00374   *(volatile uint8_t *)k_mpc_p45pfs_addr  = k_pfs_asel;
00375   *(volatile uint8_t *)k_mpc_p46pfs_addr  = k_pfs_asel;
00376   *(volatile uint8_t *)k_mpc_p47pfs_addr  = k_pfs_asel;
00377   *(volatile uint8_t *)k_mpc_pwpr_addr    = k_pwpr_unlock;
00378   *(volatile uint8_t *)k_mpc_pwpr_addr    = k_pwpr_b0wi;
00379
00380   /* 3. Configure S12AD0: 12-bit right-justified, single-scan, software trigger. */
00381   REG16(k_s12ad0_adcer_addr)   = k_adcer bit_right;
00382   REG16(k_s12ad0_adansa0_addr) = k_adansa_ch4567;
00383
00384   /* 4. Sampling state count = 20 for each of the 4 channels. */
00385   *(volatile uint8_t *)(k_s12ad0_adsstr4_addr + k_adsstr_off_ch4) = k_adsstr_def;
00386   *(volatile uint8_t *)(k_s12ad0_adsstr4_addr + k_adsstr_off_ch5) = k_adsstr_def;
00387   *(volatile uint8_t *)(k_s12ad0_adsstr4_addr + k_adsstr_off_ch6) = k_adsstr_def;
00388   *(volatile uint8_t *)(k_s12ad0_adsstr4_addr + k_adsstr_off_ch7) = k_adsstr_def;
00389
00390   /* 5. ADCSR = 0: single-scan, software trigger, no interrupt. */
00391   REG16(k_s12ad0_adcsr_addr) = 0U;
00392 }
```




```
void adc0_read_all (
                uint16_t * m0,
                uint16_t * m1,
                uint16_t * m2,
                uint16_t * m3)    [static]
```

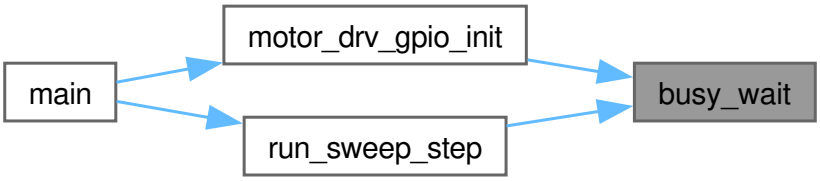
```
00396 {
00397   REG16(k_s12ad0_adcsr_addr) |= k_adcsr_adst;
00398   for (volatile uint32_t i = 0U; i < (uint32_t)k_adc_adst_poll_max; i++) {
00399     if ((REG16(k_s12ad0_adcsr_addr) & k_adcsr_adst) == 0U) { break; }
00400   }
00401   /* Per motor_spin_test k_motors[] ordering: M0 uses IN on P1/P2 ports, but
00402    * ISENSE pins are the DRV8263H IPROPI outputs wired per docs/03_hardware_pinout:
00403    *   AN007 (ADDR7) = Motor 0, AN006 (ADDR6) = Motor 1,
00404    *   AN005 (ADDR5) = Motor 2, AN004 (ADDR4) = Motor 3. */
00405   if (m0 != (uint16_t *)0) { *m0 = REG16(k_s12ad0_addr7_addr); }
00406   if (m1 != (uint16_t *)0) { *m1 = REG16(k_s12ad0_addr6_addr); }
00407   if (m2 != (uint16_t *)0) { *m2 = REG16(k_s12ad0_addr5_addr); }
```

```
00408   if (m3 != (uint16_t *)0) { *m3 = REG16(k_s12ad0_addr4_addr); }
00409 }
```



void busy_wait (
uint32_t iters) [static]

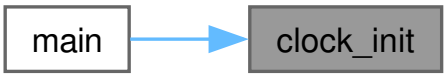
```
00206 {
00207   for (volatile uint32_t i = 0; i < iters; i++) {
00208     __asm__ volatile("nop");
00209   }
00210 }
```



void clock_init (
void) [extern]

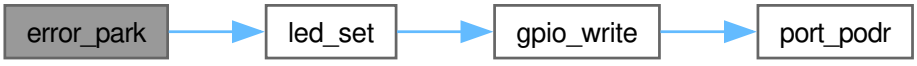
```
00083 {
00084   volatile rx_system_regs_t *sys = system_regs();
00085
00086   /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
00087   *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00088
00089   /* Stop sub-clock oscillator -- not used on STAR. */
00090   sys->sosccr = k_sub_clock_stopped;
00091
00092   /* Start MOSC (24 MHz external crystal). */
00093   sys->mosccr = k_main_osc_enabled;
00094
00095   /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
00096    * settled (typically ~10 ms physical time) instead of waiting a fixed
00097    * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
00098   for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
00099     if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
00100       break;
00101     }
00102   }
00103
00104   /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
00105   sys->pllcr  = k_pllcr_mosc_x10;
00106   sys->pllcr2 = k_pll_enabled;
00107
00108   /* Wait for PLL lock (bounded). If we time out, the crystal is missing
00109    * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
00110    * on a dead clock. Stay on LOCO so the LEDs still respond and the user
00111    * can see we got past clock_init. */
```

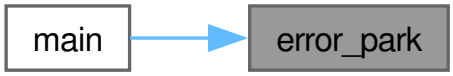
```
00112   bool pll_locked = false;
00113   for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
00114     if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
00115       pll_locked = true;
00116       break;
00117     }
00118   }
00119
00120   if (pll_locked) {
00121     /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
00122     *memwait_reg() = k_memwait_one_wait;
00123
00124     /* Apply system clock dividers. */
00125     sys->sckcr = k_sckcr_value;
00126
00127     /* Route UCLK from main PLL (UPLLSEL=0). */
00128     *((volatile uint16_t *)k_packcr_addr) = 0x0000U;
00129
00130     /* Switch system clock source to PLL. */
00131     sys->sckcr3 = k_sckcr3_cksel_pll;
00132   }
00133
00134   /* Re-lock PRCR. */
00135   *prcr_reg() = k_rx_prcr_lock;
00136 }
```



void error_park (
void) [static]

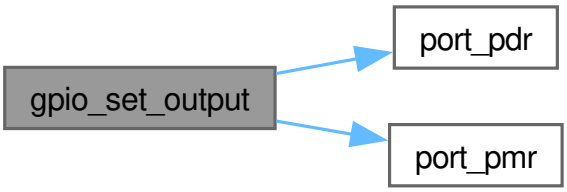
```
00471 {
00472   /* Light LED5 SOLID -- bright, visible, unmistakeable "init failed". */
00473   led_set(k_port_b, k_bit_led5, true);
00474   for (;;) { __asm__ volatile("nop"); }
00475 }
```

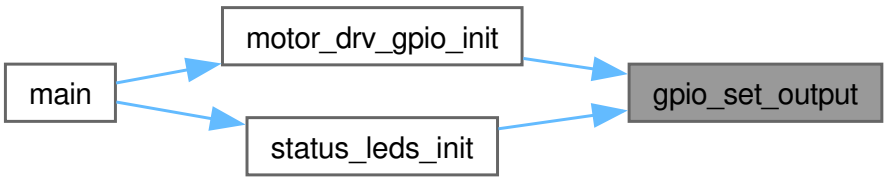




void gpio_set_output (
uint8_t port,
uint8_t pin) [inline], [static]

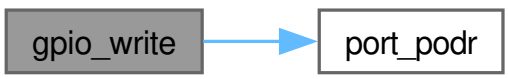
```
00216 {
00217   *port_pmr(port) &= (uint8_t) ~(1U « pin); /* peripheral mode off -> GPIO */
00218   *port_pdr(port) |= (uint8_t)(1U « pin);   /* direction = output */
00219 }
```

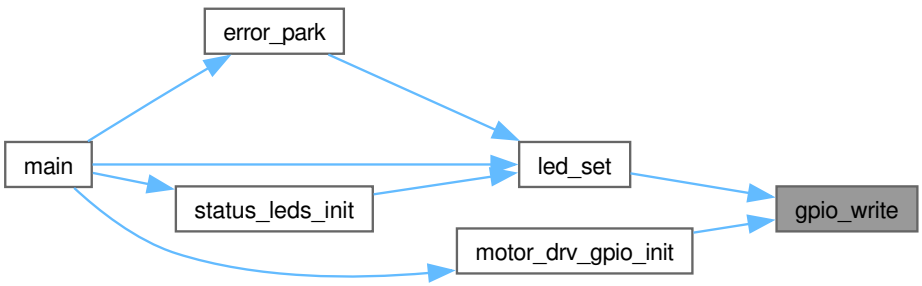




void gpio_write (
uint8_t port,
uint8_t pin,
bool high) [inline], [static]

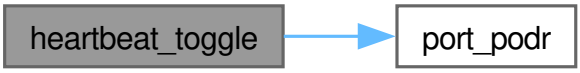
```
00222 {
00223   if (high) {
00224     *port_podr(port) |= (uint8_t)(1U « pin);
00225   } else {
00226     *port_podr(port) &= (uint8_t) ~(1U « pin);
00227   }
00228 }
```

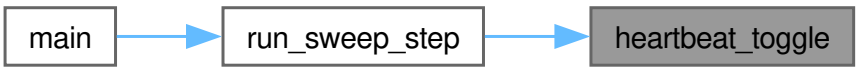




void heartbeat_toggle (
void) [static]

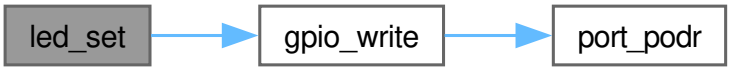
```
00466 {
00467   *port_podr(k_port_a) ^= (uint8_t)(1U « k_bit_led0);
00468 }
```

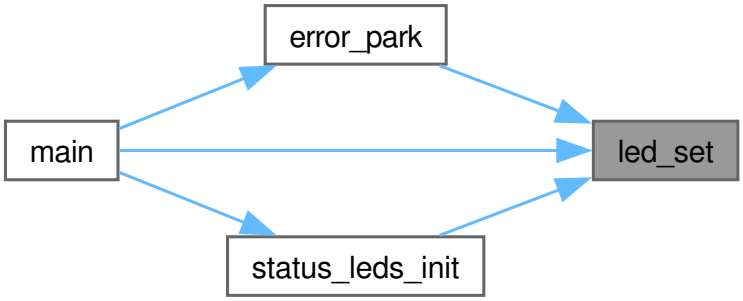




void led_set (
uint8_t port,
uint8_t bit,
bool on) [inline], [static]

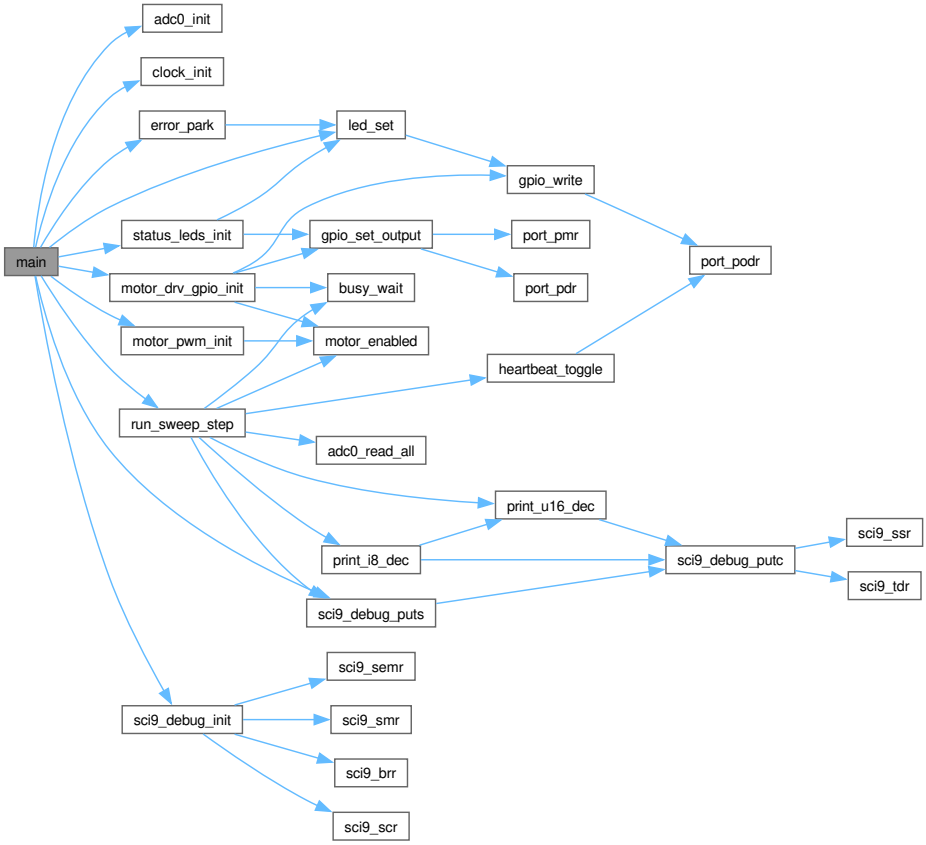
```
00450 {
00451   /* on==true -> drive output HIGH (LED lights) */
00452   gpio_write(port, bit, on);
00453 }
```





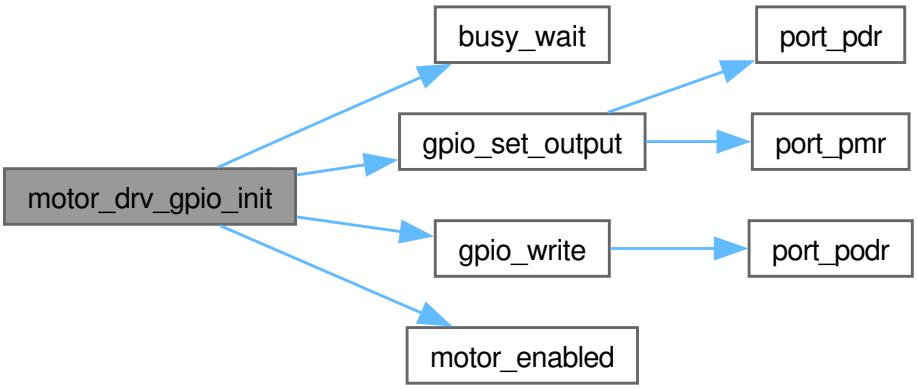
int main (
                    void)

```
00525 {
00526   /* Step 0: bring up the status LEDs FIRST -- before any clock change
00527    * so a hung clock_init still leaves a lit LED telling us "main was
00528    * entered and GPIO works". RX72N GPIO is not gated by module-stop;
00529    * direct port writes work on the default LOCO 240 kHz clock. */
00530   status_leds_init();
00531   led_set(k_port_b, k_bit_led1, true); /* LED1 = main entered */
00532
00533   /* Step 1: 24 MHz crystal -> PLL 240 MHz -> ICLK=240, PCKA=120 MHz.
00534    * If this hangs/crashes, only LED1 is lit -- tells us clock_init
00535    * is the culprit (likely PLL not locking). */
00536   clock_init();
00537   led_set(k_port , k_bit_led2, true); /* LED2 = clock_init returned */
00538
00539   /* Step 1a: bring up debug UART so every subsequent init stage reports. */
00540   sci9_debug_init();
00541   sci9_debug_puts("\r\n[motor_spin_test] boot, clocks up\r\n");
00542
00543   /* Step 1b: init S12AD0 for AN004..AN007 (motor IPROPI sense). */
00544   adc0_init();
00545   sci9_debug_puts("[motor_spin_test] S12AD0 AN004-AN007 ready\r\n");
00546
00547   /* Step 2: DRV8263H control GPIOs in the safe order
00548    * (DRVOFF HIGH -> nSLEEP HIGH -> tWAKE -> DRVOFF LOW). */
00549   motor_drv_gpio_init();
00550   led_set(k_port , k_bit_led3, true); /* LED3 = bridges enabled */
00551
00552   /* Step 3: init GPTW PWM pins + motor handles. LED4 lit confirms
00553    * rx_motor_init succeeded for every enabled channel. */
00554   static rx_motor_handle_t s_motors[k_motor_count];
00555   if (!motor_pwm_init(s_motors)) {
00556     error_park();
00557   }
00558   led_set(k_port_b, k_bit_led4, true); /* LED4 = PWM channels live */
00559
00560   /* Step 5: forever sweep duty. */
00561   int8_t duty_pc = k_duty_min;
00562   int8_t dir     = k_duty_step_pc;
00563   for (;;) {
00564     run_sweep_step(s_motors, duty_pc);
00565
00566     const int next = (int)duty_pc + (int)dir;
00567     if (next >= (int)k_duty_max) {
00568       duty_pc = k_duty_max;
00569       dir     = (int8_t)(-k_duty_step_pc);
00570     } else if (next <= (int)k_duty_min) {
00571       duty_pc = k_duty_min;
00572       dir     = (int8_t)k_duty_step_pc;
00573     } else {
00574       duty_pc = (int8_t)next;
00575     }
00576   }
00577
00578   /* Unreachable. */
00579   return 0;
00580 }
```



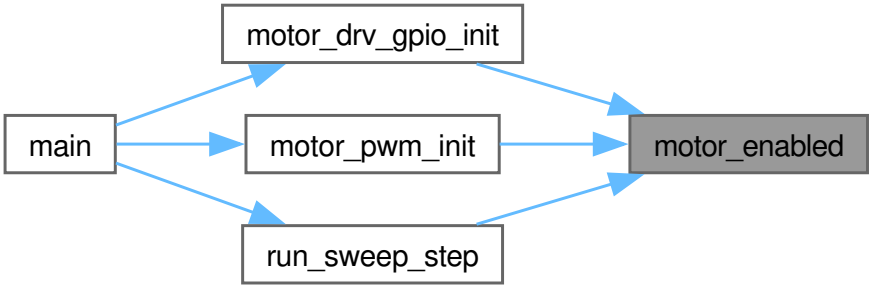
void motor_drv_gpio_init (void) [static]

```
00241 {
00242   /* Step 1+2: configure ctrl pins as outputs, DRVOFF HIGH first.
00243    * Disabled motors still get DRVOFF asserted HIGH (safe: bridge off). */
00244   for (uint8_t i = 0; i < k_motor_count; i++) {
00245     gpio_set_output(k_motors[i].drvoff_port, k_motors[i].drvoff_pin);
00246     gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, true);
00247   }
00248   for (uint8_t i = 0; i < k_motor_count; i++) {
00249     if (!motor_enabled(i)) { continue; } /* leave nSLEEP at POR default */
00250     gpio_set_output(k_motors[i].nsleep_port, k_motors[i].nsleep_pin);
00251     gpio_write(k_motors[i].nsleep_port, k_motors[i].nsleep_pin, true);
00252   }
00253
00254   /* Step 3: tWAKE busy-wait (DRV8263H needs ~1.2 ms after nSLEEP rising). */
00255   busy_wait(k_twake_iters);
00256
00257   /* Step 4: enable bridge outputs only on motors selected by the mask. */
00258   for (uint8_t i = 0; i < k_motor_count; i++) {
00259     if (!motor_enabled(i)) { continue; }
00260     gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, false);
00261   }
00262 }
```



bool motor_enabled (
uint8_t i) [inline], [static]

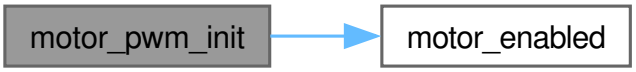
```
00138 {
00139   return (bool)(((MOTOR_ENABLE_MASK) & (1U « i)) != 0U);
00140 }
```



bool motor_pwm_init (
rx_motor_handle_t handles[k_motor_count]) [static]

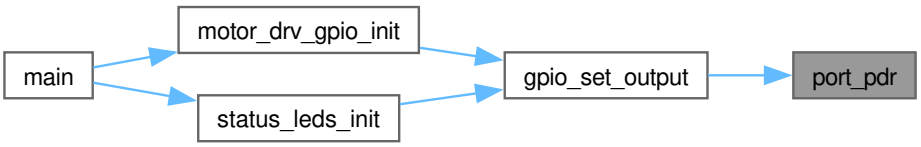
```
00270 {
00271   for (uint8_t i = 0; i < k_motor_count; i++) {
00272     if (!motor_enabled(i)) { continue; }
00273     const rx_motor_config_t cfg = {
00274       .channel      = k_motors[i].gptw_channel,
00275       .output_a     = k_gptw_output_a,          /* IN2 */
00276       .output_b     = k_gptw_output_b,          /* IN1 */
00277       .pwm_freq_hz  = 20000U,
00278       .dead_time_ns = 1000U,
00279       .invert_pwm   = false,
00280       /* Per-motor pad coordinates -- lib uses these to mux GTIOC*A/B
```

```
00281         * to the right pads. Sourced from the test's k_motors[] table
00282         * (which mirrors src/inc/hardware_config.h). */
00283        .port_a_idx   = (uint8_t)rx_port_from_pin(k_motors[i].in2_pin),
00284        .bit_a        = (uint8_t)rx_pin_from_pin(k_motors[i].in2_pin),
00285        .port_b_idx   = (uint8_t)rx_port_from_pin(k_motors[i].in1_pin),
00286        .bit_b        = (uint8_t)rx_pin_from_pin(k_motors[i].in1_pin),
00287     };
00288     if (rx_motor_init(&handles[i], &cfg) != k_rx_ok) { return false; }
00289   }
00290
00291   return true;
00292 }
```



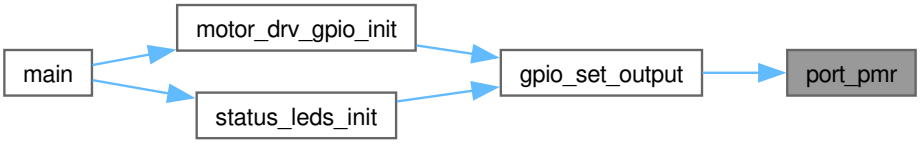
volatile uint8_t * port_pdr (
uint8_t port) [inline], [static]

```
00065 { return &REG8(k_port_pdr_base  + port); }
```



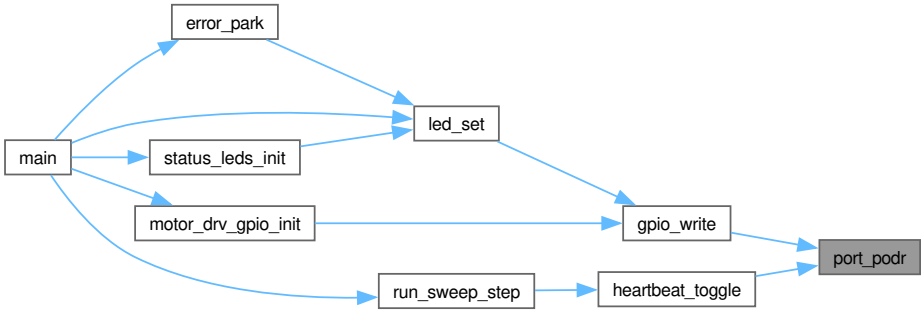
volatile uint8_t * port_pmr (
uint8_t port) [inline], [static]

```
00067 { return &REG8(k_port_pmr_base  + port); }
```



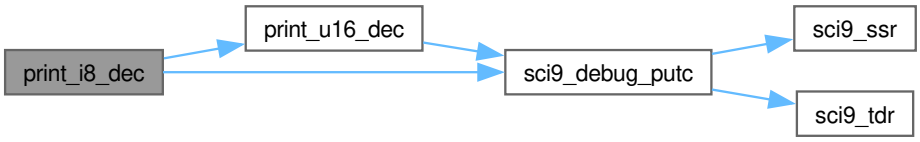
volatile uint8_t * port_podr (
uint8_t port) [inline], [static]

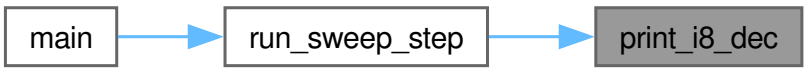
```
00066 { return &REG8(k_port_podr_base + port); }
```



void print_i8_dec (
int8_t v) [static]

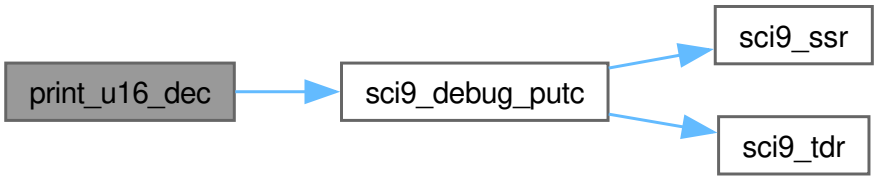
```
00431 {
00432   if (v < 0) {
00433     sci9_debug_putc('-');
00434     print_u16_dec((uint16_t)(-(int16_t)v));
00435   } else {
00436     sci9_debug_putc('+');
00437     print_u16_dec((uint16_t)v);
00438   }
00439 }
```

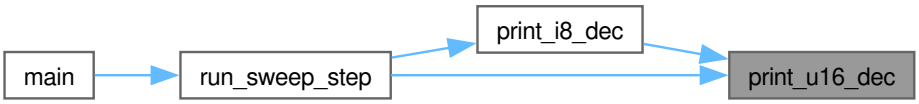




void print_u16_dec (
uint16_t v) [static]

```
00413 {
00414   char     buf[k_u16_dec_max_digits];
00415   uint8_t n = 0;
00416   if (v == 0U) {
00417     sci9_debug_putc('0');
00418     return;
00419   }
00420   while (v > 0U && n < (uint8_t)sizeof(buf)) {
00421     buf[n++] = (char)('0' + (v % k_dec_radix));
00422     v = (uint16_t)(v / k_dec_radix);
00423   }
00424   while (n > 0U) {
00425     sci9_debug_putc(buf[--n]);
00426   }
00427 }
```

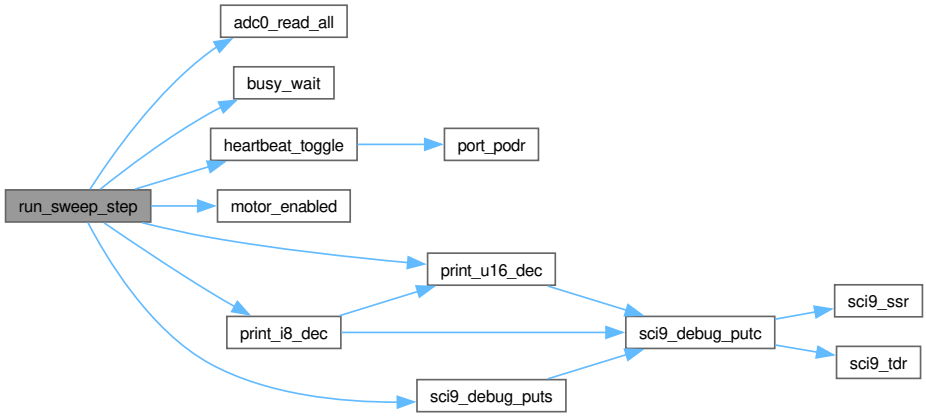




void run_sweep_step (
rx_motor_handle_t handles[k_motor_count],
int8_t duty_pc) [static]

```
00483 {
00484   for (uint8_t i = 0; i < k_motor_count; i++) {
00485     if (!motor_enabled(i)) { continue; }
00486     /* Apply per-motor direction sign so 'positive duty' always means
00487      * 'wheel rolls robot forward', regardless of how each motor is
00488      * wired. Per-motor signs live in k_motors[].direction_sign. */
00489     const int duty_signed = (int)duty_pc * (int)k_motors[i].direction_sign;
00490     (void)rx_motor_set_duty(&handles[i], (float)duty_signed);
00491   }
00492   heartbeat_toggle();
00493
00494   /* Let PWM settle, then sample IPROPI on all 4 motors and report.
00495    * Delay is split in half before/after the ADC scan so the duty step
00496    * is visible on the UART roughly mid-dwell, not at its edges. */
00497   busy_wait(k_step_delay_iters / k_step_delay_halves);
00498
00499   uint16_t adc_m0 = 0;
00500   uint16_t adc_m1 = 0;
00501   uint16_t adc_m2 = 0;
00502   uint16_t adc_m3 = 0;
00503   adc0_read_all(&adc_m0, &adc_m1, &adc_m2, &adc_m3);
00504
00505   /* Format: "D=<duty> M0=<raw> M1=<raw> M2=<raw> M3=<raw>\r\n"
```

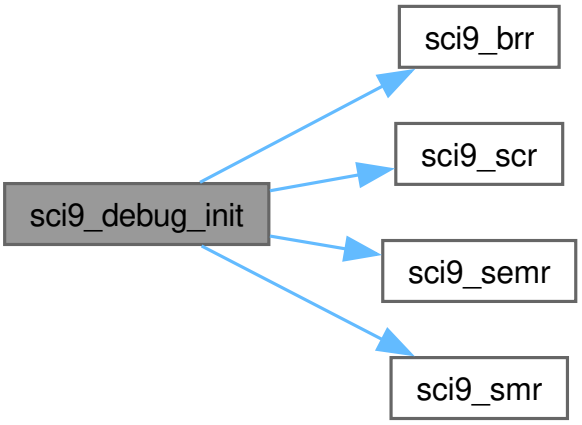
```
00506     * Raw is 12-bit ADC count 0..4095. Convert on host:
00507     *   V_mV  = raw * 3300 / 4095
00508     *   I_mA  = V_mV * 10000 / 10302   (DRV8263H: 5.1k sense, 202 uA/A mirror) */
00509    sci9_debug_puts("D=");
00510    print_i8_dec(duty_pc);
00511    sci9_debug_puts(" M0=");
00512    print_u16_dec(adc_m0);
00513    sci9_debug_puts(" M1=");
00514    print_u16_dec(adc_m1);
00515    sci9_debug_puts(" M2=");
00516    print_u16_dec(adc_m2);
00517    sci9_debug_puts(" M3=");
00518    print_u16_dec(adc_m3);
00519    sci9_debug_puts("\r\n");
00520
00521    busy_wait(k_step_delay_iters / k_step_delay_halves);
00522 }
```



void sci9_debug_init (void) [extern]

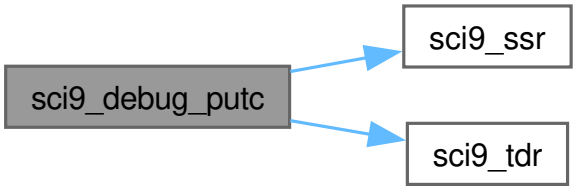
```
00105 {
00106     /* Step 1: release SCI9 from module stop. SCI9 = MSTPCRC bit 26 per
00107      * RX72N HW manual page 410 (NOT MSTPCRB.bit22 -- that's PDC). */
00108     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00109     system_regs()->mstpcrc &= ~(uint32_t)(1UL << (uint32_t)k_mstpc_sci9_bit);
00110     *prcr_reg() = k_rx_prcr_lock;
00111
00112     /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC.  Use the struct
00113      * accessor so the compiler computes the correct PFS offsets via
00114      * static_assert-checked layout. */
00115     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear B0WI */
00116     mpc()->pwpr   = k_pwpr_pfswe;     /* allow PFS writes */
00117     mpc()->pb6pfs = k_psel_sci9;
00118     mpc()->pb7pfs = k_psel_sci9;
00119     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear PFSWE */
00120     mpc()->pwpr   = k_pwpr_b0wi;      /* re-protect */
00121
00122     /* Step 3: PDR before PMR (production uart.c does this).  PB7 = output
00123      * for TX, PB6 = input for RX.  Then flip PMR to peripheral mode. */
```

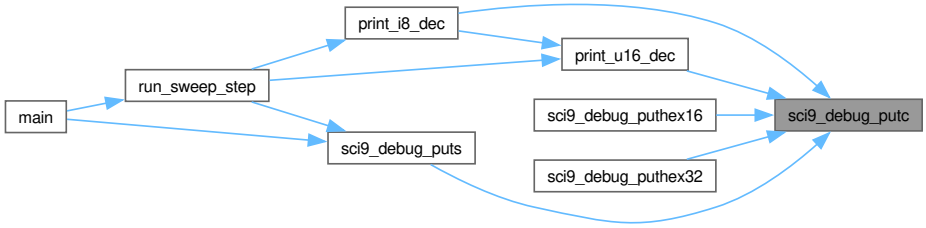
```
00124     portb()->pdr |= (uint8_t)k_pb7_mask;
00125     portb()->pdr &= (uint8_t) ~k_pb6_mask;
00126     portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);
00127
00128     /* Step 4: SCI9 init.  Match the production order:
00129      *   SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
00130      * Don't touch SCMR (reset default 0xF2 is correct). */
00131     *sci9_scr() = k_scr_disabled;
00132     *sci9_smr() = k_smr n1_async;
00133     *sci9_brr() = k_brr ;
00134
00135     /* Wait roughly one bit time before enabling TX/RX. */
00136     for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
00137         ___asm___ volatile("nop");
00138     }
00139
00140     *sci9_scr()  = k_scr_txrx_enabled;
00141     *sci9_semr() = k_semr_default;
00142 }
```



void sci9_debug_putc (
char c) [extern]

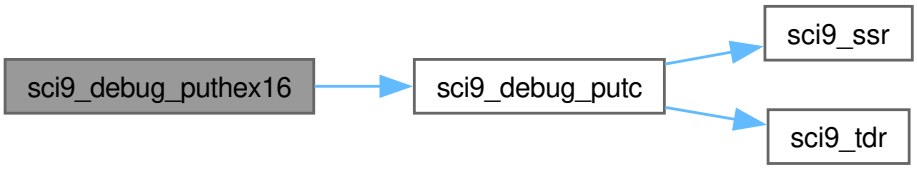
```
00148 {
00149     while ((*sci9_ssr() & (uint8_t)k_ssr_tdre) == 0U) {
00150         /* spin */
00151     }
00152     *sci9_tdr() = (uint8_t)c;
00153 }
```





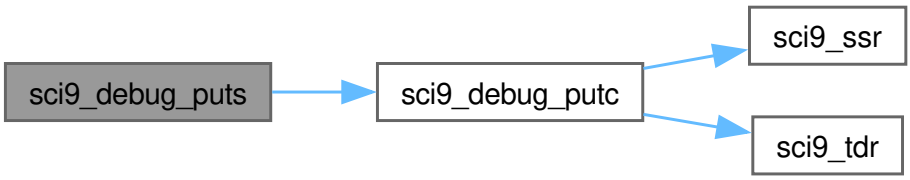
void sci9_debug_puthex16 (
uint16_t v) [extern]

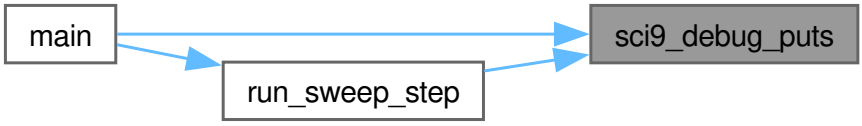
```
00180 {
00181     static const char hex_digits[] = "0123456789ABCDEF";
00182     sci9_debug_putc('0');
00183     sci9_debug_putc('x');
00184     for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
00185          shift -= (int8_t)k_hex_shift_step) {
00186         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00187     }
00188 }
```



void sci9_debug_puts (
const char * s) [extern]

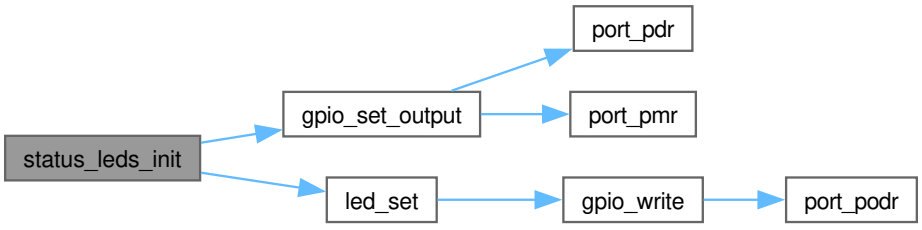
```
00156 {
00157     if (s == (const char *)0) {
00158         return;
00159     }
00160     while (*s != '\0') {
00161         if (*s == '\n') {
00162             sci9_debug_putc('\r');
00163         }
00164         sci9_debug_putc(*s++);
00165     }
00166 }
```





void status_leds_init (void) [static]

```
00456 {
00457   gpio_set_output(k_port_a, k_bit_led0); led_set(k_port_a, k_bit_led0, false);
00458   gpio_set_output(k_port_b, k_bit_led1); led_set(k_port_b, k_bit_led1, false);
00459   gpio_set_output(k_port , k_bit_led2); led_set(k_port , k_bit_led2, false);
00460   gpio_set_output(k_port , k_bit_led3); led_set(k_port , k_bit_led3, false);
00461   gpio_set_output(k_port_b, k_bit_led4); led_set(k_port_b, k_bit_led4, false);
00462   gpio_set_output(k_port_b, k_bit_led5); led_set(k_port_b, k_bit_led5, false);
00463 }
```



const motor_wiring_t k_motors[k_motor_count] [static]

= {

```
  {.gptw_channel = k_gptw_channel ,
   .in2_pin      = k_rx_p2 , .in1_pin = k_rx_p1 ,
   .drvoff_port  = k_port ,  .drvoff_pin = 1,
   .nsleep_port  = k_port ,  .nsleep_pin = 0,
   .direction_sign = -1},

  {.gptw_channel = k_gptw_channel ,
   .in2_pin      = k_rx_p2 , .in1_pin = k_rx_pc ,
   .drvoff_port  = k_port ,  .drvoff_pin = 3,
   .nsleep_port  = k_port ,  .nsleep_pin = 2,
   .direction_sign = +1},


  {.gptw_channel = k_gptw_channel ,
   .in2_pin      = k_rx_pe , .in1_pin = k_rx_p8 ,
   .drvoff_port  = k_port_e,  .drvoff_pin = 0,
   .nsleep_port  = k_port ,  .nsleep_pin = 4,
   .direction_sign = +1},


  {.gptw_channel = k_gptw_channel ,
   .in2_pin      = k_rx_pe , .in1_pin = k_rx_pc ,
   .drvoff_port  = k_port_e,  .drvoff_pin = 2,
   .nsleep_port  = k_port_e,  .nsleep_pin = 1,
   .direction_sign = -1},
}
```

```
00142                                           {
00143   /* M0 = front-left. Wired backwards on this board -- positive duty
00144    * drives the wheel in reverse, so we flip the sign here. */
00145   {.gptw_channel = k_gptw_channel ,
00146    .in2_pin      = k_rx_p2 , .in1_pin = k_rx_p1 ,
00147    .drvoff_port  = k_port ,  .drvoff_pin = 1,
00148    .nsleep_port  = k_port ,  .nsleep_pin = 0,
00149    .direction_sign = -1},
00150
00151   {.gptw_channel = k_gptw_channel ,
00152    .in2_pin      = k_rx_p2 , .in1_pin = k_rx_pc ,
00153    .drvoff_port  = k_port ,  .drvoff_pin = 3,
00154    .nsleep_port  = k_port ,  .nsleep_pin = 2,
00155    .direction_sign = +1},
00156
00157   /* M2 = back-right. */
00158   {.gptw_channel = k_gptw_channel ,
00159    .in2_pin      = k_rx_pe , .in1_pin = k_rx_p8 ,
00160    .drvoff_port  = k_port_e,  .drvoff_pin = 0,
00161    .nsleep_port  = k_port ,  .nsleep_pin = 4,
00162    .direction_sign = +1},
00163
00164   /* M3 = back-left. Wired backwards (matches M0 front-left -- both
00165    * left-side wheels are mirrored on this chassis). */
00166   {.gptw_channel = k_gptw_channel ,
00167    .in2_pin      = k_rx_pe , .in1_pin = k_rx_pc ,
00168    .drvoff_port  = k_port_e,  .drvoff_pin = 2,
00169    .nsleep_port  = k_port_e,  .nsleep_pin = 1,
00170    .direction_sign = -1},
00171 };
```

```
00001 /**
00002  * @file main.c
00003  * @brief Open-loop 4-motor spin test.
00004  *
00005  * @details
00006  * Bare-metal bench test that drives all 4 DRV8263H motor channels from the
00007  * RX72N GPTW peripheral with a -100..+100..-100 percent duty sweep. Reuses
00008  * the production motor-control libraries (rx_motor / rx_gptw / rx_mpc) and
00009  * the production crystal+PLL clock setup, but skips ThreadX, USB, SPI,
00010  * IMU, sonar, ADC, and the closed-loop PID task.
00011  *
00012  * Pin map (mirrors star-rx72n-firmware/src/inc/hardware_config.h and
00013  * libs/rx_hal/inc/hardware.h):
00014  *
00015  *   Motor 0: GPTW0 IN2=P23 (pin 34), IN1=P17 (pin 38)
```

```
00016  *           DRVOFF=P61 (pin 115), nSLEEP=P60 (pin 117)
00017  *   Motor 1: GPTW1 IN2=P22 (pin 35), IN1=PC3 (pin 67)
00018  *           DRVOFF=P63 (pin 113), nSLEEP=P62 (pin 114)
00019  *   Motor 2: GPTW2 IN2=PE3 (pin 108), IN1=P86 (pin 41)
00020  *           DRVOFF=PE0 (pin 111), nSLEEP=P64 (pin 112)
00021  *   Motor 3: GPTW3 IN2=PE7 (pin 101), IN1=PC6 (pin 61)
00022  *           DRVOFF=PE2 (pin 109), nSLEEP=PE1 (pin 110)
00023  *
00024  * Power: motors require bus voltage on the barrel/battery input.
00025  * USB power alone is NOT enough to spin them.
00026  *
00027  * Heartbeat: PA4 toggles each duty step so a third scope probe (or an LED
00028  * if one is fitted on PA4) confirms firmware is alive.
00029  *
00030  * SPDX-License-Identifier: MIT
00031  * @copyright Copyright (c) 2026 Locked Inc.
00032  */
00033
00034 #include <stdint.h>
00035 #include <stdbool.h>
00036 #include <stddef.h>
00037
00038 #include "rx_motor.h"
00039 #include "rx_gptw.h"
00040 #include "rx_port_constants.h"
00041
00042 extern void clock_init(void);
00043
00044 /* SCI9 polled debug UART (PB7=TXD, PB6=RXD) -> CY7C65213 -> /dev/ttyACM0.
00045  * Baud 115200 8N1, matches encoder_test/sci9.c verbatim. */
00046 extern void sci9_debug_init(void);
00047 extern void sci9_debug_putc(char c);
00048 extern void sci9_debug_puts(const char *s);
00049 extern void sci9_debug_puthex16(uint16_t v);
00050
00051 /*
      ============================================================================
00052  * Direct port register access -- PDR / PODR / PMR for one port n live at
00053  * 0x0008C000+n, 0x0008C020+n, 0x0008C060+n respectively (RX72N HW manual
00054  * Ch.22 I/O Ports). Same direct-register pattern as pwm_test_fit; keeps this test free
00055  * of a dependency on rx_port_utils / gpio.c.
00056  *
      ============================================================================
      */
00057 #define REG8(a) (*(volatile uint8_t *)(uintptr_t)(a))
00058
00059 typedef enum : uintptr_t {
00060   k_port_pdr_base  = 0x0008C000U,
00061   k_port_podr_base = 0x0008C020U,
00062   k_port_pmr_base  = 0x0008C060U,
00063 } port_reg_base_t;
00064
00065 static inline volatile uint8_t *port_pdr(uint8_t port)  { return &REG8(k_port_pdr_base  + port); }
00066 static inline volatile uint8_t *port_podr(uint8_t port) { return &REG8(k_port_podr_base + port); }
00067 static inline volatile uint8_t *port_pmr(uint8_t port)  { return &REG8(k_port_pmr_base  + port); }
00068
00069 /*
00070  * Status LED map:
00071  *   LED0 = PA7  (pin 88) -- heartbeat (toggles each duty step)
00072  *   LED1 = PB0  (pin 87) -- lit after clock_init
00073  *   LED2 = P71  (pin 86) -- lit after motor_drv_gpio_init
00074  *   LED3 = P72  (pin 85) -- lit after motor_pwm_init succeeds
00075  *   LED4 = PB1  (pin 84) -- lit if any motor has |duty| > 0 right now
00076  *   LED5 = PB2  (pin 83) -- lit SOLID on error_park (init failed)
00077  *
00078  * If you see LED1+LED2+LED3 on and LED0 toggling -> firmware is healthy
00079  * and PWM is being driven. If LED5 is on, rx_motor_init failed.
00080  */
00081 typedef enum : uint8_t {
00082   k_port_a   = 10, /* PORTA */
00083   k_port_b   = 11, /* PORTB */
00084   k_port     = 7,  /* PORT7 */
00085   k_bit_led0 = 7,  /* PA7 */
00086   k_bit_led1 = 0,  /* PB0 */
00087   k_bit_led2 = 1,  /* P71 */
00088   k_bit_led3 = 2,  /* P72 */
00089   k_bit_led4 = 1,  /* PB1 */
00090   k_bit_led5 = 2,  /* PB2 */
00091 } status_led_pins_t;
00092
00093 /* RX72N port indices used by the motor map. PORT0-PORT7 share their numeric
00094  * index; PORT8=8, PORTC=12, PORTE=14. Match rx_port_utils convention. */
00095 typedef enum : uint8_t {
00096   k_port  = 1,
00097   k_port  = 2,
00098   k_port  = 6,
00099   k_port  = 8,
```

```
00100   k_port_c = 12,
00101   k_port_e = 14,
00102 } rx_port_index_t;
00103
00104 /*
      ============================================================================
00105  * Per-motor wiring table -- mirrors src/inc/hardware_config.h
00106  *
      ============================================================================
      */
00107 typedef struct {
00108   rx_gptw_channel_t gptw_channel; /* GPTW channel 0..3 */
00109   rx_port_pin_t     in2_pin;      /* GTIOC*A pin (DRV8263H IN2 / direction) */
00110   rx_port_pin_t     in1_pin;      /* GTIOC*B pin (DRV8263H IN1 / PWM) */
00111   uint8_t           drvoff_port;  /* port index of DRVOFF GPIO */
00112   uint8_t           drvoff_pin;   /* bit position 0..7 in that port */
00113   uint8_t           nsleep_port;
00114   uint8_t           nsleep_pin;
00115   int8_t            direction_sign; /* +1 = motor wired so positive duty drives
00116                                        robot forward; -1 = wired backwards
00117                                        (test multiplies sweep duty by this). */
00118 } motor_wiring_t;
00119
00120 typedef enum : uint8_t {
00121   k_motor_count = 4,
00122 } motor_count_t;
00123
00124 /*
00125  * MOTOR_ENABLE_MASK -- compile-time bitmask selecting which motors run.
00126  *   bit 0 = Motor 0, bit 1 = Motor 1, bit 2 = Motor 2, bit 3 = Motor 3.
00127  *
00128  * Default 0xF spins all four motors (matches the original test). Override
00129  * from the build system (e.g. -DMOTOR_ENABLE_MASK=0x1 for Motor 0 only).
00130  * Disabled motors are completely skipped: their PWM/GPIO is never touched
00131  * and their DRVOFF stays HIGH (driver outputs disabled, safe).
00132  */
00133 #ifndef MOTOR_ENABLE_MASK
00134 #define MOTOR_ENABLE_MASK 0xFU
00135 #endif
00136
00137 static inline bool motor_enabled(uint8_t i)
00138 {
00139   return (bool)(((MOTOR_ENABLE_MASK) & (1U << i)) != 0U);
00140 }
00141
00142 static const motor_wiring_t k_motors[k_motor_count] = {
00143   /* M0 = front-left. Wired backwards on this board -- positive duty
00144    * drives the wheel in reverse, so we flip the sign here. */
00145   {.gptw_channel = k_gptw_channel ,
00146    .in2_pin      = k_rx_p2 , .in1_pin = k_rx_p1 ,
00147    .drvoff_port  = k_port ,  .drvoff_pin = 1,
00148    .nsleep_port  = k_port ,  .nsleep_pin = 0,
00149    .direction_sign = -1},
00150
00151   {.gptw_channel = k_gptw_channel ,
00152    .in2_pin      = k_rx_p2 , .in1_pin = k_rx_pc ,
00153    .drvoff_port  = k_port ,  .drvoff_pin = 3,
00154    .nsleep_port  = k_port ,  .nsleep_pin = 2,
00155    .direction_sign = +1},
00156
00157   /* M2 = back-right. */
00158   {.gptw_channel = k_gptw_channel ,
00159    .in2_pin      = k_rx_pe , .in1_pin = k_rx_p8 ,
00160    .drvoff_port  = k_port_e,  .drvoff_pin = 0,
00161    .nsleep_port  = k_port ,  .nsleep_pin = 4,
00162    .direction_sign = +1},
00163
00164   /* M3 = back-left. Wired backwards (matches M0 front-left -- both
00165    * left-side wheels are mirrored on this chassis). */
00166   {.gptw_channel = k_gptw_channel ,
00167    .in2_pin      = k_rx_pe , .in1_pin = k_rx_pc ,
00168    .drvoff_port  = k_port_e,  .drvoff_pin = 2,
00169    .nsleep_port  = k_port_e,  .nsleep_pin = 1,
00170    .direction_sign = -1},
00171 };
00172
00173 /*
      ============================================================================
00174  * Sweep timing -- bounded loops per NASA Rule 2
00175  *
      ============================================================================
      */
00176 typedef enum : uint32_t {
00177   /* ~40 ms per duty step at ICLK=240 MHz, -O1; tuned coarsely. */
00178   k_step_delay_iters = 1000000U,
00179   /* ~10 ms tWAKE margin after nSLEEP rises (DRV8263H spec: 1.2 ms min). */
00180   k_twake_iters = 250000U,
```

```
00181   /* Split each duty-step dwell around the ADC scan (half before, half after). */
00182   k_step_delay_halves = 2U,
00183 } delay_iters_t;
00184
00185 /*
00186  * MOTOR_DUTY_MIN / MOTOR_DUTY_MAX -- compile-time override of the duty
00187  * sweep range. Defaults to -100..+100 (bidirectional). Override for the
00188  * forward-only / reverse-only build targets:
00189  *    Forward only: -DMOTOR_DUTY_MIN=0    -DMOTOR_DUTY_MAX=100
00190  *    Reverse only: -DMOTOR_DUTY_MIN=-100 -DMOTOR_DUTY_MAX=0
00191  */
00192 #ifndef MOTOR_DUTY_MIN
00193 #define MOTOR_DUTY_MIN (-100)
00194 #endif
00195 #ifndef MOTOR_DUTY_MAX
00196 #define MOTOR_DUTY_MAX (100)
00197 #endif
00198
00199 typedef enum : int8_t {
00200   k_duty_min     = MOTOR_DUTY_MIN,
00201   k_duty_max     = MOTOR_DUTY_MAX,
00202   k_duty_step_pc = 5,
00203 } duty_sweep_t;
00204
00205 static void busy_wait(uint32_t iters)
00206 {
00207   for (volatile uint32_t i = 0; i < iters; i++) {
00208     __asm__ volatile("nop");
00209   }
00210 }
00211
00212 /*
      ============================================================================
00213  * GPIO control of the DRV8263H DRVOFF / nSLEEP signals
00214  *
      ============================================================================
      */
00215 static inline void gpio_set_output(uint8_t port, uint8_t pin)
00216 {
00217   *port_pmr(port) &= (uint8_t) ~(1U << pin); /* peripheral mode off -> GPIO */
00218   *port_pdr(port) |= (uint8_t)(1U << pin);   /* direction = output */
00219 }
00220
00221 static inline void gpio_write(uint8_t port, uint8_t pin, bool high)
00222 {
00223   if (high) {
00224     *port_podr(port) |= (uint8_t)(1U << pin);
00225   } else {
00226     *port_podr(port) &= (uint8_t) ~(1U << pin);
00227   }
00228 }
00229
00230 /*
      ============================================================================
00231  * Init the DRV8263H control GPIOs in the safe order:
00232  *    1. DRVOFF HIGH first  -> H-bridge outputs disabled
00233  *    2. nSLEEP HIGH second -> driver wakes up with bridge already disabled
00234  *    3. busy-wait tWAKE
00235  *    4. DRVOFF LOW         -> bridge outputs enabled, ready for PWM
00236  *
00237  * Ordering follows DRV8263H datasheet sec 7.3.1 and matches
00238  * internal_gpio_init_motor_driver_ctrl() in src/hardware_init.c.
00239  *
      ============================================================================
      */
00240 static void motor_drv_gpio_init(void)
00241 {
00242   /* Step 1+2: configure ctrl pins as outputs, DRVOFF HIGH first.
00243    * Disabled motors still get DRVOFF asserted HIGH (safe: bridge off). */
00244   for (uint8_t i = 0; i < k_motor_count; i++) {
00245     gpio_set_output(k_motors[i].drvoff_port, k_motors[i].drvoff_pin);
00246     gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, true);
00247   }
00248   for (uint8_t i = 0; i < k_motor_count; i++) {
00249     if (!motor_enabled(i)) { continue; } /* leave nSLEEP at POR default */
00250     gpio_set_output(k_motors[i].nsleep_port, k_motors[i].nsleep_pin);
00251     gpio_write(k_motors[i].nsleep_port, k_motors[i].nsleep_pin, true);
00252   }
00253
00254   /* Step 3: tWAKE busy-wait (DRV8263H needs ~1.2 ms after nSLEEP rising). */
00255   busy_wait(k_twake_iters);
00256
00257   /* Step 4: enable bridge outputs only on motors selected by the mask. */
00258   for (uint8_t i = 0; i < k_motor_count; i++) {
00259     if (!motor_enabled(i)) { continue; }
00260     gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, false);
00261   }
```

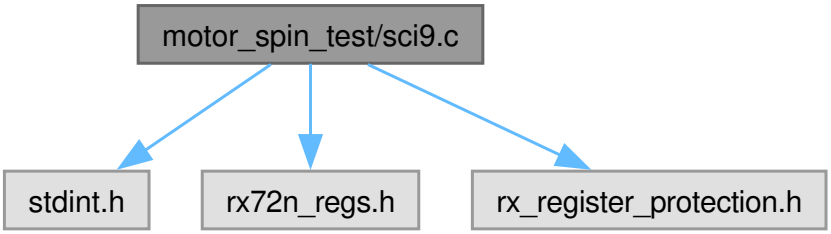
```
00262 }
00263
00264 /*
      ============================================================================
00265  * Bring up GPTW pins via the production MPC helper, then init each motor.
00266  * Returns false on any error -- caller should park the heartbeat LED on
00267  * solid as a visual indication of init failure.
00268  *
      ============================================================================
      */
00269 static bool motor_pwm_init(rx_motor_handle_t handles[k_motor_count])
00270 {
00271   for (uint8_t i = 0; i < k_motor_count; i++) {
00272     if (!motor_enabled(i)) { continue; }
00273     const rx_motor_config_t cfg = {
00274       .channel      = k_motors[i].gptw_channel,
00275       .output_a     = k_gptw_output_a,          /* IN2 */
00276       .output_b     = k_gptw_output_b,          /* IN1 */
00277       .pwm_freq_hz  = 20000U,
00278       .dead_time_ns = 1000U,
00279       .invert_pwm   = false,
00280       /* Per-motor pad coordinates -- lib uses these to mux GTIOC*A/B
00281        * to the right pads. Sourced from the test's k_motors[] table
00282        * (which mirrors src/inc/hardware_config.h). */
00283       .port_a_idx   = (uint8_t)rx_port_from_pin(k_motors[i].in2_pin),
00284       .bit_a        = (uint8_t)rx_pin_from_pin(k_motors[i].in2_pin),
00285       .port_b_idx   = (uint8_t)rx_port_from_pin(k_motors[i].in1_pin),
00286       .bit_b        = (uint8_t)rx_pin_from_pin(k_motors[i].in1_pin),
00287     };
00288     if (rx_motor_init(&handles[i], &cfg) != k_rx_ok) { return false; }
00289   }
00290
00291   return true;
00292 }
00293
00294 /*
      ============================================================================
00295  * S12AD0 (12-bit ADC) bare-metal driver -- reads AN004-AN007 on P44-P47
00296  * (DRV8263H IPROPI for motors 3,2,1,0 respectively).
00297  *
00298  *   S12AD0 base = 0x00089000
00299  *   ADCSR    @ +0x00 (16b)  - start/mode control
00300  *   ADANSA0  @ +0x04 (16b)  - channel enable mask AN000..AN015
00301  *   ADCER    @ +0x0E (16b)  - resolution/data format
00302  *   ADSSTR4..7 @ +0xE4..0xE7 (8b each) - sampling state count
00303  *   ADDR4..7 @ +0x28..0x2E (16b each) - conversion results
00304  *
00305  *   MSTPCRA.MSTPA17 releases S12AD0 from module-stop.
00306  *   PFS.ASEL=1 turns P44-P47 into analog inputs.
00307  *
      ============================================================================
      */
00308 #define REG16(a) (*(volatile uint16_t *)(uintptr_t)(a))
00309
00310 typedef enum : uintptr_t {
00311   k_s12ad0_adcsr_addr   = 0x00089000U,
00312   k_s12ad0_adansa0_addr = 0x00089004U,
00313   k_s12ad0_adcer_addr   = 0x0008900EU,
00314   k_s12ad0_adsstr4_addr = 0x000890E4U,
00315   k_s12ad0_addr4_addr   = 0x00089028U, /* AN004 result */
00316   k_s12ad0_addr5_addr   = 0x0008902AU,
00317   k_s12ad0_addr6_addr   = 0x0008902CU,
00318   k_s12ad0_addr7_addr   = 0x0008902EU,
00319   k_mstpcra_addr        = 0x00080010U,
00320   k_prcr_addr           = 0x000803FEU,
00321   k_mpc_pwpr_addr       = 0x0008C11FU,
00322   k_mpc_p44pfs_addr     = 0x0008C124U,
00323   k_mpc_p45pfs_addr     = 0x0008C125U,
00324   k_mpc_p46pfs_addr     = 0x0008C126U,
00325   k_mpc_p47pfs_addr     = 0x0008C127U,
00326 } adc_addrs_t;
00327
00328 typedef enum : uint16_t {
00329   k_adcsr_adst        = (uint16_t)(1U << 15),
00330   k_adansa_ch4567     = (uint16_t)0x00F0U, /* enable AN004..AN007 */
00331   k_adcer_bit_right = (uint16_t)0x0000U, /* ADPRC=00, ADRFMT=0 */
00332   k_prcr_unlock       = (uint16_t)0xA50BU, /* unlock PRC0+PRC1+PRC3 */
00333   k_prcr_lock         = (uint16_t)0xA500U,
00334 } adc_cfg16_t;
00335
00336 typedef enum : uint32_t {
00337   k_mstpa17 = (uint32_t)(1UL << 17), /* S12AD0 stop bit */
00338 } adc_mstp_t;
00339
00340 typedef enum : uint8_t {
00341   k_pfs_asel     = 0x80U,
00342   k_pwpr_unlock  = 0x00U,
```

```
00343   k_pwpr_pfswe    = 0x40U,
00344   k_pwpr_b0wi     = 0x80U,
00345   k_adsstr_def    = 0x14U, /* 20 states sampling, conservative */
00346   /* ADSSTR channel offsets in bytes from ADSSTR4 base (one per channel). */
00347   k_adsstr_off_ch4 = 0U,
00348   k_adsstr_off_ch5 = 1U,
00349   k_adsstr_off_ch6 = 2U,
00350   k_adsstr_off_ch7 = 3U,
00351   /* Decimal print helpers. */
00352   k_u16_dec_max_digits = 5U,  /* 65535 is 5 digits */
00353   k_dec_radix          = 10U,
00354 } adc_cfg8_t;
00355
00356 typedef enum : uint32_t {
00357   /* Bound on polling loop waiting for ADCSR.ADST to self-clear; the actual
00358    * conversion completes in <5 us even at 60 MHz PCLKB, so 100k iters is
00359    * pure safety against a hung peripheral (~400 us wall-clock @ 240 MHz). */
00360   k_adc_adst_poll_max = 100000U,
00361 } adc_poll_t;
00362
00363 static void adc0_init(void)
00364 {
00365   /* 1. Release S12AD0 from module-stop (MSTPA17 in MSTPCRA). */
00366   REG16(k_prcr_addr) = k_prcr_unlock;
00367   *(volatile uint32_t *)k_mstpcra_addr &= ~(uint32_t)k_mstpa17;
00368   REG16(k_prcr_addr) = k_prcr_lock;
00369
00370   /* 2. Set PFS.ASEL=1 for P44-P47 (analog input, digital buffer off). */
00371   *(volatile uint8_t *)k_mpc_pwpr_addr    = k_pwpr_unlock;
00372   *(volatile uint8_t *)k_mpc_pwpr_addr    = k_pwpr_pfswe;
00373   *(volatile uint8_t *)k_mpc_p44pfs_addr  = k_pfs_asel;
00374   *(volatile uint8_t *)k_mpc_p45pfs_addr  = k_pfs_asel;
00375   *(volatile uint8_t *)k_mpc_p46pfs_addr  = k_pfs_asel;
00376   *(volatile uint8_t *)k_mpc_p47pfs_addr  = k_pfs_asel;
00377   *(volatile uint8_t *)k_mpc_pwpr_addr    = k_pwpr_unlock;
00378   *(volatile uint8_t *)k_mpc_pwpr_addr    = k_pwpr_b0wi;
00379
00380   /* 3. Configure S12AD0: 12-bit right-justified, single-scan, software trigger. */
00381   REG16(k_s12ad0_adcer_addr)   = k_adcer bit_right;
00382   REG16(k_s12ad0_adansa0_addr) = k_adansa_ch4567;
00383
00384   /* 4. Sampling state count = 20 for each of the 4 channels. */
00385   *(volatile uint8_t *)(k_s12ad0_adsstr4_addr + k_adsstr_off_ch4) = k_adsstr_def;
00386   *(volatile uint8_t *)(k_s12ad0_adsstr4_addr + k_adsstr_off_ch5) = k_adsstr_def;
00387   *(volatile uint8_t *)(k_s12ad0_adsstr4_addr + k_adsstr_off_ch6) = k_adsstr_def;
00388   *(volatile uint8_t *)(k_s12ad0_adsstr4_addr + k_adsstr_off_ch7) = k_adsstr_def;
00389
00390   /* 5. ADCSR = 0: single-scan, software trigger, no interrupt. */
00391   REG16(k_s12ad0_adcsr_addr) = 0U;
00392 }
00393
00394 /* Single-shot scan of all 4 channels. Blocks until conversion completes. */
00395 static void adc0_read_all(uint16_t *m0, uint16_t *m1, uint16_t *m2, uint16_t *m3)
00396 {
00397   REG16(k_s12ad0_adcsr_addr) |= k_adcsr_adst;
00398   for (volatile uint32_t i = 0U; i < (uint32_t)k_adc_adst_poll_max; i++) {
00399     if ((REG16(k_s12ad0_adcsr_addr) & k_adcsr_adst) == 0U) { break; }
00400   }
00401   /* Per motor_spin_test k_motors[] ordering: M0 uses IN on P1/P2 ports, but
00402    * ISENSE pins are the DRV8263H IPROPI outputs wired per docs/03_hardware_pinout:
00403    *   AN007 (ADDR7) = Motor 0, AN006 (ADDR6) = Motor 1,
00404    *   AN005 (ADDR5) = Motor 2, AN004 (ADDR4) = Motor 3. */
00405   if (m0 != (uint16_t *)0) { *m0 = REG16(k_s12ad0_addr7_addr); }
00406   if (m1 != (uint16_t *)0) { *m1 = REG16(k_s12ad0_addr6_addr); }
00407   if (m2 != (uint16_t *)0) { *m2 = REG16(k_s12ad0_addr5_addr); }
00408   if (m3 != (uint16_t *)0) { *m3 = REG16(k_s12ad0_addr4_addr); }
00409 }
00410
00411 /* Print a uint16_t as up-to-5-digit decimal (no padding). */
00412 static void print_u16_dec(uint16_t v)
00413 {
00414   char    buf[k_u16_dec_max_digits];
00415   uint8_t n = 0;
00416   if (v == 0U) {
00417     sci9_debug_putc('0');
00418     return;
00419   }
00420   while (v > 0U && n < (uint8_t)sizeof(buf)) {
00421     buf[n++] = (char)('0' + (v % k_dec_radix));
00422     v = (uint16_t)(v / k_dec_radix);
00423   }
00424   while (n > 0U) {
00425     sci9_debug_putc(buf[--n]);
00426   }
00427 }
00428
00429 /* Print a signed int8 as decimal with leading sign. */
```

```
00430 static void print_i8_dec(int8_t v)
00431 {
00432   if (v < 0) {
00433     sci9_debug_putc('-');
00434     print_u16_dec((uint16_t)(-(int16_t)v));
00435   } else {
00436     sci9_debug_putc('+');
00437     print_u16_dec((uint16_t)v);
00438   }
00439 }
00440
00441 /*
      ============================================================================
00442  * Status LED helpers (production board)
00443  *
      ============================================================================
00444  *
00445  * NOTE: All 6 status LEDs are *active-high*: driving the GPIO HIGH
00446  * sources current through the LED. Silk-screen reference: D10 = LED1 (PB0,
00447  * pin 87); full 6-LED map at the top of this file.
00448  */
00449 static inline void led_set(uint8_t port, uint8_t bit, bool on)
00450 {
00451   /* on==true -> drive output HIGH (LED lights) */
00452   gpio_write(port, bit, on);
00453 }
00454
00455 static void status_leds_init(void)
00456 {
00457   gpio_set_output(k_port_a, k_bit_led0); led_set(k_port_a, k_bit_led0, false);
00458   gpio_set_output(k_port_b, k_bit_led1); led_set(k_port_b, k_bit_led1, false);
00459   gpio_set_output(k_port , k_bit_led2); led_set(k_port , k_bit_led2, false);
00460   gpio_set_output(k_port , k_bit_led3); led_set(k_port , k_bit_led3, false);
00461   gpio_set_output(k_port_b, k_bit_led4); led_set(k_port_b, k_bit_led4, false);
00462   gpio_set_output(k_port_b, k_bit_led5); led_set(k_port_b, k_bit_led5, false);
00463 }
00464
00465 static void heartbeat_toggle(void)
00466 {
00467   *port_podr(k_port_a) ^= (uint8_t)(1U « k_bit_led0);
00468 }
00469
00470 static void error_park(void)
00471 {
00472   /* Light LED5 SOLID -- bright, visible, unmistakeable "init failed". */
00473   led_set(k_port_b, k_bit_led5, true);
00474   for (;;) { ___asm___ volatile("nop"); }
00475 }
00476
00477 /*
      ============================================================================
00478  * Duty sweep: walk -100 -> +100 -> -100 in 5% steps on all 4 motors at once.
00479  * Bounded outer loop because it lives inside an unbounded `for(;;)` -- this
00480  * is the same pattern as rx72n_main task loops.
00481  *
      ============================================================================
      */
00482 static void run_sweep_step(rx_motor_handle_t handles[k_motor_count], int8_t duty_pc)
00483 {
00484   for (uint8_t i = 0; i < k_motor_count; i++) {
00485     if (!motor_enabled(i)) { continue; }
00486     /* Apply per-motor direction sign so 'positive duty' always means
00487      * 'wheel rolls robot forward', regardless of how each motor is
00488      * wired. Per-motor signs live in k_motors[].direction_sign. */
00489     const int duty_signed = (int)duty_pc * (int)k_motors[i].direction_sign;
00490     (void)rx_motor_set_duty(&handles[i], (float)duty_signed);
00491   }
00492   heartbeat_toggle();
00493
00494   /* Let PWM settle, then sample IPROPI on all 4 motors and report.
00495    * Delay is split in half before/after the ADC scan so the duty step
00496    * is visible on the UART roughly mid-dwell, not at its edges. */
00497   busy_wait(k_step_delay_iters / k_step_delay_halves);
00498
00499   uint16_t adc_m0 = 0;
00500   uint16_t adc_m1 = 0;
00501   uint16_t adc_m2 = 0;
00502   uint16_t adc_m3 = 0;
00503   adc0_read_all(&adc_m0, &adc_m1, &adc_m2, &adc_m3);
00504
00505   /* Format: "D=<duty> M0=<raw> M1=<raw> M2=<raw> M3=<raw>\r\n"
00506    * Raw is 12-bit ADC count 0..4095. Convert on host:
00507    *   V_mV  = raw * 3300 / 4095
00508    *   I_mA  = V_mV * 10000 / 10302   (DRV8263H: 5.1k sense, 202 uA/A mirror) */
00509   sci9_debug_puts("D=");
00510   print_i8_dec(duty_pc);
00511   sci9_debug_puts(" M0=");
```

```
00512   print_u16_dec(adc_m0);
00513   sci9_debug_puts(" M1=");
00514   print_u16_dec(adc_m1);
00515   sci9_debug_puts(" M2=");
00516   print_u16_dec(adc_m2);
00517   sci9_debug_puts(" M3=");
00518   print_u16_dec(adc_m3);
00519   sci9_debug_puts("\r\n");
00520
00521   busy_wait(k_step_delay_iters / k_step_delay_halves);
00522 }
00523
00524 int main(void)
00525 {
00526   /* Step 0: bring up the status LEDs FIRST -- before any clock change
00527    * so a hung clock_init still leaves a lit LED telling us "main was
00528    * entered and GPIO works". RX72N GPIO is not gated by module-stop;
00529    * direct port writes work on the default LOCO 240 kHz clock. */
00530   status_leds_init();
00531   led_set(k_port_b, k_bit_led1, true); /* LED1 = main entered */
00532
00533   /* Step 1: 24 MHz crystal -> PLL 240 MHz -> ICLK=240, PCKA=120 MHz.
00534    * If this hangs/crashes, only LED1 is lit -- tells us clock_init
00535    * is the culprit (likely PLL not locking). */
00536   clock_init();
00537   led_set(k_port , k_bit_led2, true); /* LED2 = clock_init returned */
00538
00539   /* Step 1a: bring up debug UART so every subsequent init stage reports. */
00540   sci9_debug_init();
00541   sci9_debug_puts("\r\n[motor_spin_test] boot, clocks up\r\n");
00542
00543   /* Step 1b: init S12AD0 for AN004..AN007 (motor IPROPI sense). */
00544   adc0_init();
00545   sci9_debug_puts("[motor_spin_test] S12AD0 AN004-AN007 ready\r\n");
00546
00547   /* Step 2: DRV8263H control GPIOs in the safe order
00548    * (DRVOFF HIGH -> nSLEEP HIGH -> tWAKE -> DRVOFF LOW). */
00549   motor_drv_gpio_init();
00550   led_set(k_port , k_bit_led3, true); /* LED3 = bridges enabled */
00551
00552   /* Step 3: init GPTW PWM pins + motor handles. LED4 lit confirms
00553    * rx_motor_init succeeded for every enabled channel. */
00554   static rx_motor_handle_t s_motors[k_motor_count];
00555   if (!motor_pwm_init(s_motors)) {
00556     error_park();
00557   }
00558   led_set(k_port_b, k_bit_led4, true); /* LED4 = PWM channels live */
00559
00560   /* Step 5: forever sweep duty. */
00561   int8_t duty_pc = k_duty_min;
00562   int8_t dir     = k_duty_step_pc;
00563   for (;;) {
00564     run_sweep_step(s_motors, duty_pc);
00565
00566     const int next = (int)duty_pc + (int)dir;
00567     if (next >= (int)k_duty_max) {
00568       duty_pc = k_duty_max;
00569       dir     = (int8_t)(-k_duty_step_pc);
00570     } else if (next <= (int)k_duty_min) {
00571       duty_pc = k_duty_min;
00572       dir     = (int8_t)k_duty_step_pc;
00573     } else {
00574       duty_pc = (int8_t)next;
00575     }
00576   }
00577
00578   /* Unreachable. */
00579   return 0;
00580 }
```

```
#include <stdint.h>
  #include "rx72n_regs.h"
  #include "rx_register_protection.h"
```

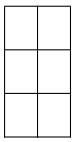


<<<<

*
*
*
*
*
*
*

*

enum hex_shifts_t : uint8_t

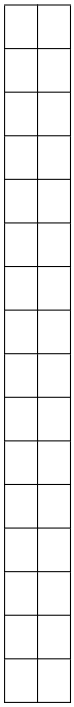


```
00062             : uint8_t {
00063     k_hex_shift_init32 = 28U,
00064     k_hex_shift_init16 = 12U,
00065     k_hex_shift_step   = 4U,
00066 } hex_shifts_t;
```

enum sci9_addrs_t : uintptr_t

```
00035             : uintptr_t {
00036     k_sci9_base = 0x000D0020U,
00037 } sci9_addrs_t;
```

enum sci9_cfg_t : uint8_t



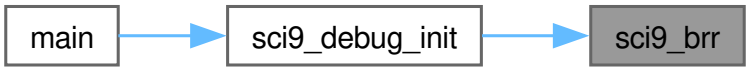
```
00039             : uint8_t {
00040     k_psel_sci9        = 0x0AU, /**< SCI9 RXD/TXD function */
00041     k_pb6_mask         = (uint8_t)(1U « 6U),
00042     k_pb7_mask         = (uint8_t)(1U « 7U),
00043     k_mstpc_sci9_bit   = 26U, /* MSTPCRC.MSTPC26 per RX72N HW manual page 410 */
00044     k_pwpr_unlock_b0   = 0x00U,
00045     k_pwpr_pfswe       = 0x40U,
00046     k_pwpr_b0wi        = 0x80U,
00047     k_brr        = 31U, /* SCI9 uses PCLKA=120 MHz (SCI7-11 group); BRR=31 @ CKS=0 ABCS=0 -> ~117 kbaud
        (1.7% over 115200) */
00048     k_smr n1_async     = 0x00U,
00049     k_scr_te_only      = 0x20U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=0 */
00050     k_scr_txrx_enabled = 0x30U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=1 */
00051     k_scr_disabled     = 0x00U,
00052     k_ssr_tdre         = 0x80U,
00053     k_scmr_default     = 0xF2U,
00054     k_semr_default     = 0x00U,
00055     k_hex_nibble_mask  = 0x0FU,
00056 } sci9_cfg_t;
```

enum sci9_settle_t : uint16_t

```
00058             : uint16_t {
00059     k_brr_settle_loops = 1000U, /**< Match production uart.c (~8.68 us @ 115200) */
00060 } sci9_settle_t;
```

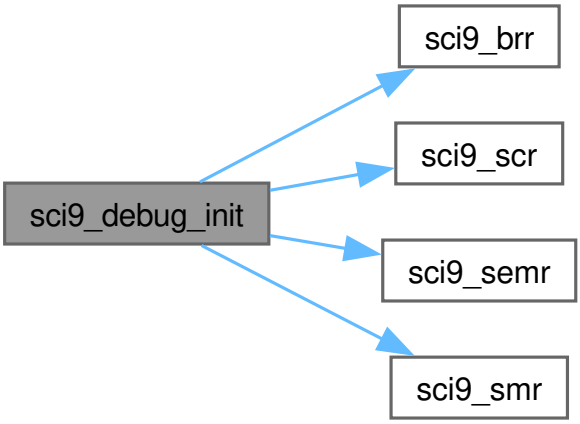
volatile uint8_t * sci9_brr (
void) [inline], [static]

```
00077 {
00078     return (volatile uint8_t *)(k_sci9_base + 1U);
00079 }
```



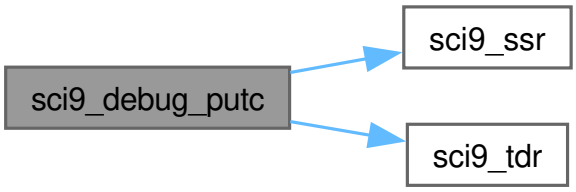
void sci9_debug_init (
void)

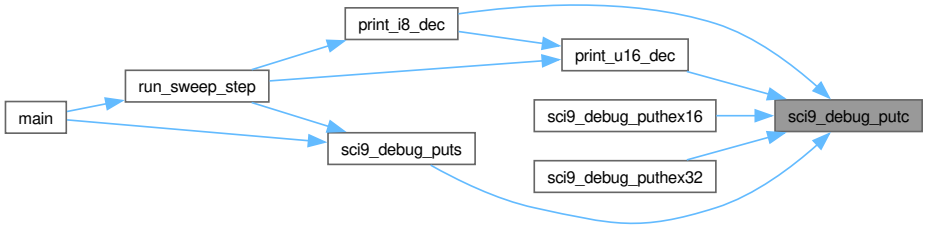
```
00105 {
00106     /* Step 1: release SCI9 from module stop. SCI9 = MSTPCRC bit 26 per
00107      * RX72N HW manual page 410 (NOT MSTPCRB.bit22 -- that's PDC). */
00108     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00109     system_regs()->mstpcrc &= ~(uint32_t)(1UL << (uint32_t)k_mstpc_sci9_bit);
00110     *prcr_reg() = k_rx_prcr_lock;
00111
00112     /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC.  Use the struct
00113      * accessor so the compiler computes the correct PFS offsets via
00114      * static_assert-checked layout. */
00115     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear B0WI */
00116     mpc()->pwpr   = k_pwpr_pfswe;     /* allow PFS writes */
00117     mpc()->pb6pfs = k_psel_sci9;
00118     mpc()->pb7pfs = k_psel_sci9;
00119     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear PFSWE */
00120     mpc()->pwpr   = k_pwpr_b0wi;      /* re-protect */
00121
00122     /* Step 3: PDR before PMR (production uart.c does this).  PB7 = output
00123      * for TX, PB6 = input for RX.  Then flip PMR to peripheral mode. */
00124     portb()->pdr |= (uint8_t)k_pb7_mask;
00125     portb()->pdr &= (uint8_t) ~k_pb6_mask;
00126     portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);
00127
00128     /* Step 4: SCI9 init.  Match the production order:
00129      *   SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
00130      * Don't touch SCMR (reset default 0xF2 is correct). */
00131     *sci9_scr() = k_scr_disabled;
00132     *sci9_smr() = k_smr n1_async;
00133     *sci9_brr() = k_brr ;
00134
00135     /* Wait roughly one bit time before enabling TX/RX. */
00136     for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
00137         __asm__ volatile("nop");
00138     }
00139
00140     *sci9_scr()  = k_scr_txrx_enabled;
00141     *sci9_semr() = k_semr_default;
00142 }
```



void sci9_debug_putc (
char c)

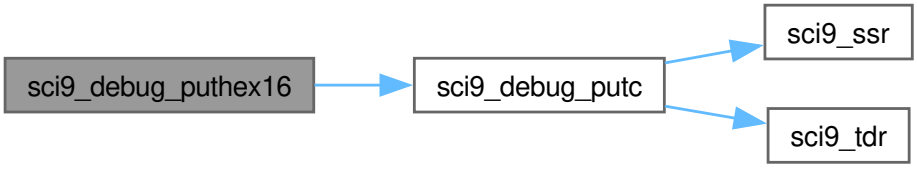
```
00148 {
00149     while ((*sci9_ssr() & (uint8_t)k_ssr_tdre) == 0U) {
00150         /* spin */
00151     }
00152     *sci9_tdr() = (uint8_t)c;
00153 }
```





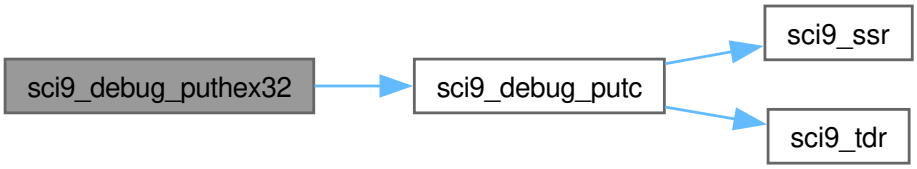
void sci9_debug_puthex16 (
uint16_t v)

```
00180 {
00181     static const char hex_digits[] = "0123456789ABCDEF";
00182     sci9_debug_putc('0');
00183     sci9_debug_putc('x');
00184     for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
00185          shift -= (int8_t)k_hex_shift_step) {
00186         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00187     }
00188 }
```



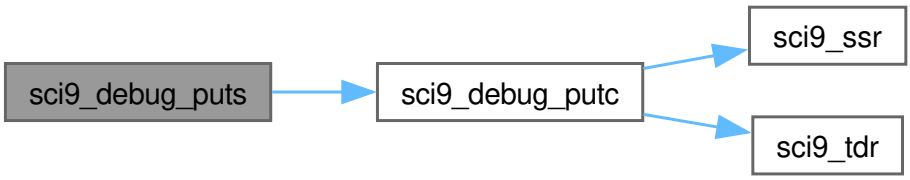
void sci9_debug_puthex32 (
uint32_t v)

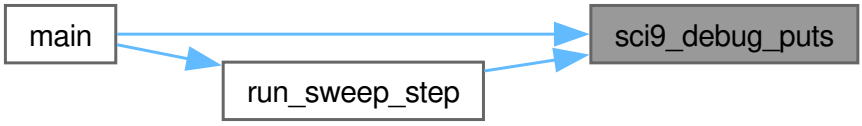
```
00169 {
00170     static const char hex_digits[] = "0123456789ABCDEF";
00171     sci9_debug_putc('0');
00172     sci9_debug_putc('x');
00173     for (int8_t shift = (int8_t)k_hex_shift_init32; shift >= 0;
00174          shift -= (int8_t)k_hex_shift_step) {
00175         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00176     }
00177 }
```



void sci9_debug_puts (
const char * s)

```
00156 {
00157     if (s == (const char *)0) {
00158         return;
00159     }
00160     while (*s != '\0') {
00161         if (*s == '\n') {
00162             sci9_debug_putc('\r');
00163         }
00164         sci9_debug_putc(*s++);
00165     }
00166 }
```



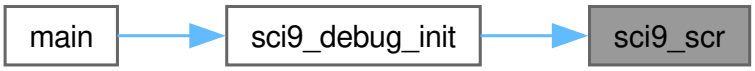


volatile uint8_t * sci9_scmr (void) [inline], [static]

```
00093 {
00094     return (volatile uint8_t *)(k_sci9_base + 6U);
00095 }
```

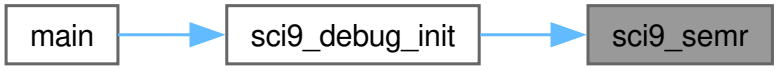
volatile uint8_t * sci9_scr (void) [inline], [static]

```
00081 {
00082     return (volatile uint8_t *)(k_sci9_base + 2U);
00083 }
```



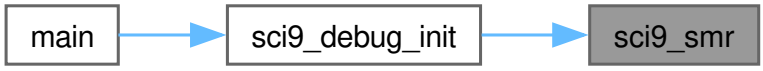
volatile uint8_t * sci9_semr (void) [inline], [static]

```
00097 {
00098     return (volatile uint8_t *)(k_sci9_base + 7U);
00099 }
```



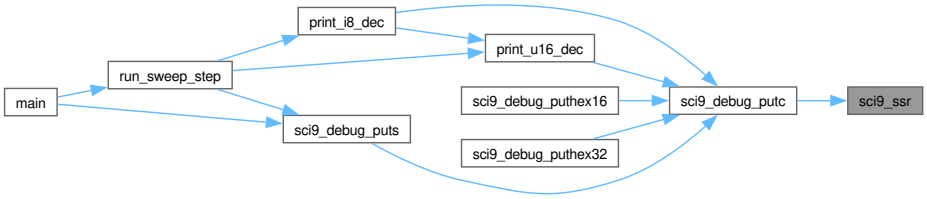
volatile uint8_t * sci9_smr (
void) [inline], [static]

```
00073 {
00074     return (volatile uint8_t *)(k_sci9_base + 0U);
00075 }
```



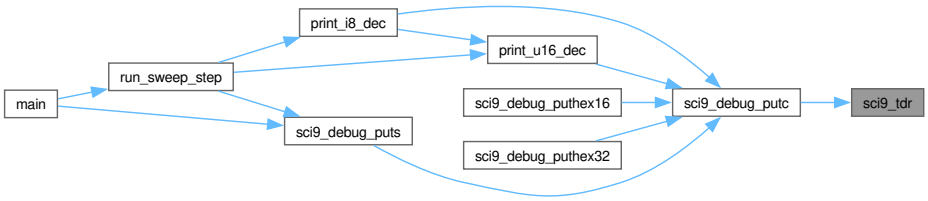
volatile uint8_t * sci9_ssr (
void) [inline], [static]

```
00089 {
00090     return (volatile uint8_t *)(k_sci9_base + 4U);
00091 }
```



volatile uint8_t * sci9_tdr (
void) [inline], [static]

```
00085 {
00086     return (volatile uint8_t *)(k_sci9_base + 3U);
00087 }
```

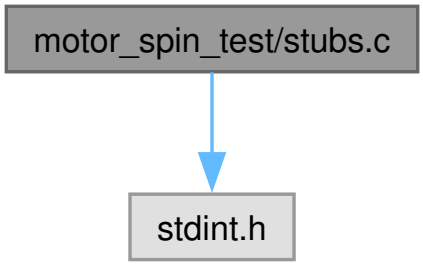


```
00001 /**
00002  * @file sci9_debug.c
00003  * @brief Minimal polled-mode SCI9 (TXD9 = PB7, RXD9 = PB6) debug UART for motor_spin_test.
00004  *
00005  * @details
00006  * The STAR board routes SCI9 to the on-board CY7C65213 USB-to-UART bridge
00007  * which appears as /dev/ttyACM0 on the host (or COMx on Windows).  This
00008  * file provides just enough init + putc/puts to emit ASCII status from
00009  * the firmware so we can debug the USB CDC bring-up without needing the
00010  * USB device to actually enumerate.
00011  *
00012  * Uses the rx_hal struct accessors (mpc(), portb(), system_regs(), prcr_reg())
00013  * so the addresses are all compiler-verified via static_assert.
00014  *
00015  * Baud = 115200. SCI9 is in the SCI7-11 group that uses PCLKA (120 MHz),
00016  * not PCLKB. At CKS=0, ABCS=0:
00017  *   BRR = (PCLKA / (32 * baud)) - 1 = 120000000 / 3686400 - 1 = 31.55 -> 31
00018  *   Actual baud = PCLKA / (32 * (BRR+1)) = 117187 Hz (+1.7% vs 115200, in tolerance).
00019  *
00020  * SPDX-License-Identifier: MIT
00021  * @copyright Copyright (c) 2026 Locked Inc.
00022  */
00023
00024 #include <stdint.h>
00025
00026 #include "rx72n_regs.h"
00027 #include "rx_register_protection.h"
00028
00029 /*
      ================================================================================
00030  * SCI9 register access via raw addresses (rx72n_sci_regs.h doesn't expose
00031  * the per-channel struct as cleanly as we'd like for a quick polled driver).
00032  * The base 0x000D0020 is taken from the verified header.
00033  * Register offsets per RX72N HW manual: SMR=0, BRR=1, SCR=2, TDR=3, SSR=4.
00034  *
      ================================================================================
      */
00035 typedef enum : uintptr_t {
00036     k_sci9_base = 0x000D0020U,
00037 } sci9_addrs_t;
00038
00039 typedef enum : uint8_t {
00040     k_psel_sci9        = 0x0AU, /**< SCI9 RXD/TXD function */
00041     k_pb6_mask         = (uint8_t)(1U << 6U),
00042     k_pb7_mask         = (uint8_t)(1U << 7U),
00043     k_mstpc_sci9_bit   = 26U, /* MSTPCRC.MSTPC26 per RX72N HW manual page 410 */
00044     k_pwpr_unlock_b0   = 0x00U,
00045     k_pwpr_pfswe       = 0x40U,
00046     k_pwpr_b0wi        = 0x80U,
00047     k_brr        = 31U, /* SCI9 uses PCLKA=120 MHz (SCI7-11 group); BRR=31 @ CKS=0 ABCS=0 -> ~117 kbaud
      (1.7% over 115200) */
00048     k_smr_n1_async     = 0x00U,
00049     k_scr_te_only      = 0x20U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=0 */
00050     k_scr_txrx_enabled = 0x30U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=1 */
00051     k_scr_disabled     = 0x00U,
00052     k_ssr_tdre         = 0x80U,
00053     k_scmr_default     = 0xF2U,
00054     k_semr_default     = 0x00U,
00055     k_hex_nibble_mask  = 0x0FU,
00056 } sci9_cfg_t;
00057
00058 typedef enum : uint16_t {
00059     k_brr_settle_loops = 1000U, /**< Match production uart.c (~8.68 us @ 115200) */
00060 } sci9_settle_t;
00061
00062 typedef enum : uint8_t {
00063     k_hex_shift_init32 = 28U,
00064     k_hex_shift_init16 = 12U,
00065     k_hex_shift_step   = 4U,
00066 } hex_shifts_t;
00067
00068 /*
      ================================================================================
00069  * SCI9 register accessors (raw addresses since the header doesn't give us a
00070  * per-channel struct for the SCIi/SCIj quirks).
00071  *
      ================================================================================
      */
00072 static inline volatile uint8_t *sci9_smr(void)
00073 {
00074     return (volatile uint8_t *)(k_sci9_base + 0U);
00075 }
00076 static inline volatile uint8_t *sci9_brr(void)
00077 {
00078     return (volatile uint8_t *)(k_sci9_base + 1U);
00079 }
```

```
00080 static inline volatile uint8_t *sci9__scr(void)
00081 {
00082     return (volatile uint8_t *)(k__sci9__base + 2U);
00083 }
00084 static inline volatile uint8_t *sci9__tdr(void)
00085 {
00086     return (volatile uint8_t *)(k__sci9__base + 3U);
00087 }
00088 static inline volatile uint8_t *sci9__ssr(void)
00089 {
00090     return (volatile uint8_t *)(k__sci9__base + 4U);
00091 }
00092 static inline volatile uint8_t *sci9__scmr(void)
00093 {
00094     return (volatile uint8_t *)(k__sci9__base + 6U);
00095 }
00096 static inline volatile uint8_t *sci9__semr(void)
00097 {
00098     return (volatile uint8_t *)(k__sci9__base + 7U);
00099 }
00100
00101 /*
    ============================================================================
00102  * Initialise SCI9 for 115200 8N1 polled output on PB7 (TXD9).
00103  *
    ============================================================================
    */
00104 void sci9__debug__init(void)
00105 {
00106     /* Step 1: release SCI9 from module stop. SCI9 = MSTPCRC bit 26 per
00107      * RX72N HW manual page 410 (NOT MSTPCRB.bit22 -- that's PDC). */
00108     *prcr__reg() = k__rx__prcr__unlock__prc0__prc1;
00109     system__regs()->mstpcrc &= ~(uint32__t)(1UL « (uint32__t)k__mstpc__sci9__bit);
00110     *prcr__reg() = k__rx__prcr__lock;
00111
00112     /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC.  Use the struct
00113      * accessor so the compiler computes the correct PFS offsets via
00114      * static_assert-checked layout. */
00115     mpc()->pwpr   = k__pwpr__unlock__b0; /* clear B0WI */
00116     mpc()->pwpr   = k__pwpr__pfswe;      /* allow PFS writes */
00117     mpc()->pb6pfs = k__psel__sci9;
00118     mpc()->pb7pfs = k__psel__sci9;
00119     mpc()->pwpr   = k__pwpr__unlock__b0; /* clear PFSWE */
00120     mpc()->pwpr   = k__pwpr__b0wi;       /* re-protect */
00121
00122     /* Step 3: PDR before PMR (production uart.c does this).  PB7 = output
00123      * for TX, PB6 = input for RX.  Then flip PMR to peripheral mode. */
00124     portb()->pdr |= (uint8__t)k__pb7__mask;
00125     portb()->pdr &= (uint8__t) ~k__pb6__mask;
00126     portb()->pmr |= (uint8__t)(k__pb6__mask | k__pb7__mask);
00127
00128     /* Step 4: SCI9 init.  Match the production order:
00129      *   SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
00130      * Don't touch SCMR (reset default 0xF2 is correct). */
00131     *sci9__scr() = k__scr__disabled;
00132     *sci9__smr() = k__smr n1__async;
00133     *sci9__brr() = k__brr ;
00134
00135     /* Wait roughly one bit time before enabling TX/RX. */
00136     for (volatile uint32__t i = 0U; i < (uint32__t)k__brr__settle__loops; i++) {
00137         ___asm___ volatile("nop");
00138     }
00139
00140     *sci9__scr()  = k__scr__txrx__enabled;
00141     *sci9__semr() = k__semr__default;
00142 }
00143
00144 /*
    ============================================================================
00145  * Polled write: spin until TDRE then drop byte into TDR.
00146  *
    ============================================================================
    */
00147 void sci9__debug__putc(char c)
00148 {
00149     while ((*sci9__ssr() & (uint8__t)k__ssr__tdre) == 0U) {
00150         /* spin */
00151     }
00152     *sci9__tdr() = (uint8__t)c;
00153 }
00154
00155 void sci9__debug__puts(const char *s)
00156 {
00157     if (s == (const char *)0) {
00158         return;
00159     }
00160     while (*s != '\0') {
```

```
00161         if (*s == '\n') {
00162             sci9_debug_putc('\r');
00163         }
00164         sci9_debug_putc(*s++);
00165     }
00166 }
00167
00168 void sci9_debug_puthex32(uint32_t v)
00169 {
00170     static const char hex_digits[] = "0123456789ABCDEF";
00171     sci9_debug_putc('0');
00172     sci9_debug_putc('x');
00173     for (int8_t shift = (int8_t)k_hex_shift_init32; shift >= 0;
00174          shift -= (int8_t)k_hex_shift_step) {
00175         sci9_debug_putc(hex_digits[(v >> (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00176     }
00177 }
00178
00179 void sci9_debug_puthex16(uint16_t v)
00180 {
00181     static const char hex_digits[] = "0123456789ABCDEF";
00182     sci9_debug_putc('0');
00183     sci9_debug_putc('x');
00184     for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
00185          shift -= (int8_t)k_hex_shift_step) {
00186         sci9_debug_putc(hex_digits[(v >> (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00187     }
00188 }
```

#include <stdint.h>



*

*

```
void rx_log_uart_putc (
                char c)


 00020 { (void)c; }

void rx_log_uart_puthex (
                uint32_t v,
                uint8_t d)


 00024 { (void)v; (void)d; }

void rx_log_uart_putint (
                int32_t v)


 00022 { (void)v; }

void rx_log_uart_puts (
                const char * s)


 00021 { (void)s; }

void rx_log_uart_putuint (
                uint32_t v)


 00023 { (void)v; }

void uart_debug_putc (
                char c)


 00027 { (void)c; }

void uart_debug_puthex (
                uint32_t v,
                uint8_t d)


 00031 { (void)v; (void)d; }

void uart_debug_putint (
                int32_t v)


 00029 { (void)v; }

void uart_debug_puts (
                const char * s)


 00028 { (void)s; }
```

void uart_debug_putuint (
uint32_t v)

00030 { (void)v; }

```
00001 /**
00002  * @file stubs.c
00003  * @brief No-op log/UART stubs for the motor spin test.
00004  *
00005  * @details
00006  * The production rx_motor / rx_gptw / rx_mpc libraries call rx_log_error()
00007  * and RX_ASSERT() macros that ultimately reduce to a handful of UART log
00008  * primitives. This bench test does not bring up SCI9 or the framed UART
00009  * ring buffer, so we satisfy the linker with no-op stubs. RX_ASSERT
00010  * failures still halt (the static-inline internal_rx_fatal_error in
00011  * rx_check.h disables interrupts and spins).
00012  *
00013  * SPDX-License-Identifier: MIT
00014  * @copyright Copyright (c) 2026 Locked Inc.
00015  */
00016
00017 #include <stdint.h>
00018
00019 /* rx_log_uart family (selected when RX_IS_SIMULATOR == 0). */
00020 void rx_log_uart_putc(char c)              { (void)c; }
00021 void rx_log_uart_puts(const char *s)       { (void)s; }
00022 void rx_log_uart_putint(int32_t v)         { (void)v; }
00023 void rx_log_uart_putuint(uint32_t v)       { (void)v; }
00024 void rx_log_uart_puthex(uint32_t v, uint8_t d) { (void)v; (void)d; }
00025
00026 /* uart_debug family (called unconditionally by internal_rx_fatal_error). */
00027 void uart_debug_putc(char c)               { (void)c; }
00028 void uart_debug_puts(const char *s)        { (void)s; }
00029 void uart_debug_putint(int32_t v)          { (void)v; }
00030 void uart_debug_putuint(uint32_t v)        { (void)v; }
00031 void uart_debug_puthex(uint32_t v, uint8_t d) { (void)v; (void)d; }
```